DHBW
Duale Hochschule
Baden-Württemberg

PC-Part-Picker

Tuluhan Engin, Fabio Schehr, Henri Weber

Datum: 04.05.26

1. Einführung - Funktion der Applikation

- Konfigurieren von PC’s
- Kompatibilitätsprüfung der Komponenten
- Speichern von Entwürfen und fertigen PC‘s
- Benchmarks durchführen

1.1 Start der Applikation

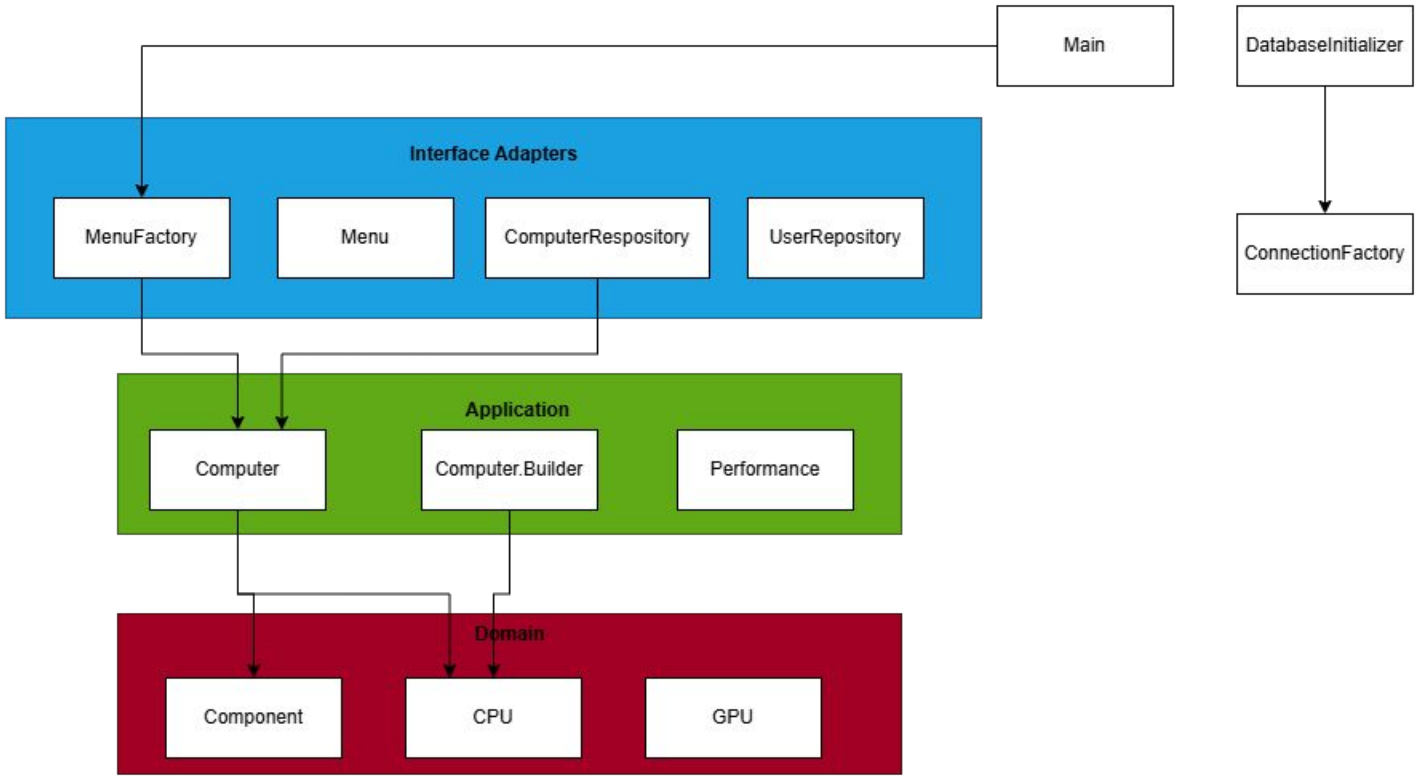
- Voraussetzungen:
  - Java 17 oder höher
  - Maven installiert

- Terminal im „tinf23b3-pcpartpicker\“ Ordner öffnen
- Startbefehl (Maven):

1.2 Technischer Überblick

- Programmiersprache: Java
- Build-Tool: Maven
- Datenbankmanagement: SQLite
- Einlesen der Seed-Daten (JSONL): Gson
- Testen: Junit 5

2. Softwarearchitektur – Clean Architecture



2.1 Domain Code

- Repräsentiert reines Fachwissen ohne Abhängigkeiten

CPU.java

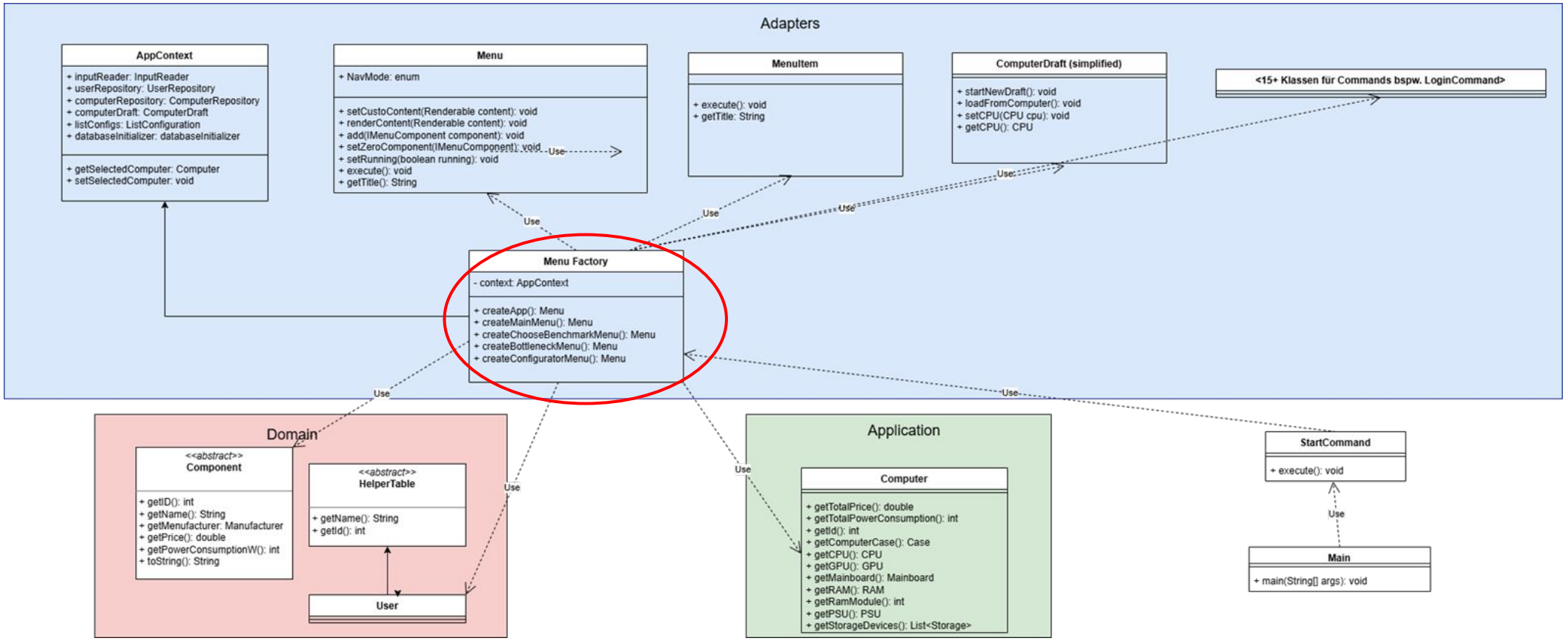
```java
/**
 * Klasse für die CPU-Komponente.
 * @param id Eindeutige ID der CPU
 * @param name Name der CPU
 * @param price Preis der CPU
 * @param manufacturer Hersteller der CPU
 * @param socket Sockeltyp der CPU (z.B. AM4, LGA1200)
 * @param speedGHz Taktfrequenz der CPU in GHz
 * @param boostClockGHz optionale Boost-Frequenz der CPU in GHz
 * @param hasIntegratedGraphics Gibt an, ob die CPU integrierte Grafik hat
 * @param powerConsumptionW Maximaler Stromverbrauch der Komponente in Watt
 * @author Fabio
 */
public class CPU extends Component {

    private final Socket socket;
    private final double speedGHz;
    private final Double boostClockGHz;
    private final boolean hasIntegratedGraphics;

    public CPU(int id, String name, double price, Manufacturer manufacturer, Socket socket, double speedGHz, boolean hasIntegratedGraphics, int powerConsumptionW) {
        this(id, name, price, manufacturer, socket, speedGHz, null, hasIntegratedGraphics, powerConsumptionW);
    }

    public CPU(int id, String name, double price, Manufacturer manufacturer, Socket socket, double speedGHz, Double boostClockGHz, boolean hasIntegratedGraphics, int powerConsumptionW) {
        super(id, name, price, manufacturer, powerConsumptionW);
        this.socket = socket;
        this.speedGHz = speedGHz;
        this.boostClockGHz = boostClockGHz;
        this.hasIntegratedGraphics = hasIntegratedGraphics;
    }

    public Socket getSocket() {
        return socket;
    }

    public double getSpeedGHz() {
        return speedGHz;
    }

    public Double getBoostClockGHz() {
        return boostClockGHz;
    }

    public boolean hasIntegratedGraphics() {
        return hasIntegratedGraphics;
```

2.2 Dependency Rule - Positiv

MenuFactory.java

```java
package de.ase.pcpartpicker.adapters.cli;

import java.util.List;
import java.util.function.BiConsumer;
import java.util.function.Supplier;

import de.ase.pcpartpicker.ColorConstants;
import de.ase.pcpartpicker.adapters.cli.commands.AutomaticFixCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ComputerToBenchmarkCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ComputerToCheckCommand;
import de.ase.pcpartpicker.adapters.cli.commands.FinishComputerCommand;
import de.ase.pcpartpicker.adapters.cli.commands.LoginCommand;
import de.ase.pcpartpicker.adapters.cli.commands.LogoutCommand;
import de.ase.pcpartpicker.adapters.cli.commands.NewUserCommand;
import de.ase.pcpartpicker.adapters.cli.commands.OpenMenuCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ResetDatabaseCommand;
import de.ase.pcpartpicker.adapters.cli.commands.RunBenchmarkCommand;
import de.ase.pcpartpicker.adapters.cli.commands.RunBottleneckCheckCommand;
import de.ase.pcpartpicker.adapters.cli.commands.SaveDraftCommand;
import de.ase.pcpartpicker.adapters.cli.commands.SelectComponentCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ShowAllUserCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ShowComputerCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ShowCurrentDraftCommand;
import de.ase.pcpartpicker.adapters.cli.commands.ShowListCommand;
import de.ase.pcpartpicker.adapters.cli.commands.StartComputerDraftCommand;
import de.ase.pcpartpicker.adapters.cli.utils.NavigationUtils;
import de.ase.pcpartpicker.domain.Component;
import de.ase.pcpartpicker.domain.HDD;
import de.ase.pcpartpicker.domain.HelperClasses.User;
import de.ase.pcpartpicker.domain.M2SSD;
import de.ase.pcpartpicker.domain.SSD;
import de.ase.pcpartpicker.part_assembly.Computer;

/**
 * Klasse, die die Menüs erstellt und konfiguriert.
 * Bezieht alle Abhängigkeiten über den AppContext.
 */
public class MenuFactory {


    private final AppContext context;

    public MenuFactory(AppContext context) {
        this.context = context;
    }

    public static Menu createApp() {
        AppContext context = new AppContext();
        return new MenuFactory(context).createMainMenu();
    }

    public Menu createMainMenu() {
        Menu mainMenu = new Menu("PC Part Picker - Hauptmenü", context.inputReader);
        mainMenu.add(new MenuItem("Verfügbare Teile ansehen", new OpenMenuCommand(createComponentMenu())));
        mainMenu.add(new MenuItem("Computerverwaltung", new OpenMenuCommand(createComputerMenu())));
        mainMenu.add(new MenuItem("Nutzerverwaltung", new OpenMenuCommand(createUserMenu())));
        mainMenu.add(new MenuItem("Login", new OpenMenuCommand(createLoginMenu())));
        mainMenu.add(new MenuItem("Datenbank reset", new ResetDatabaseCommand(context.inputReader, context.databaseInitializer)));

        NavigationUtils.addExitNavigation(mainMenu);
        return mainMenu;
    }

```

2.2 Dependency Rule – Positiv - UML



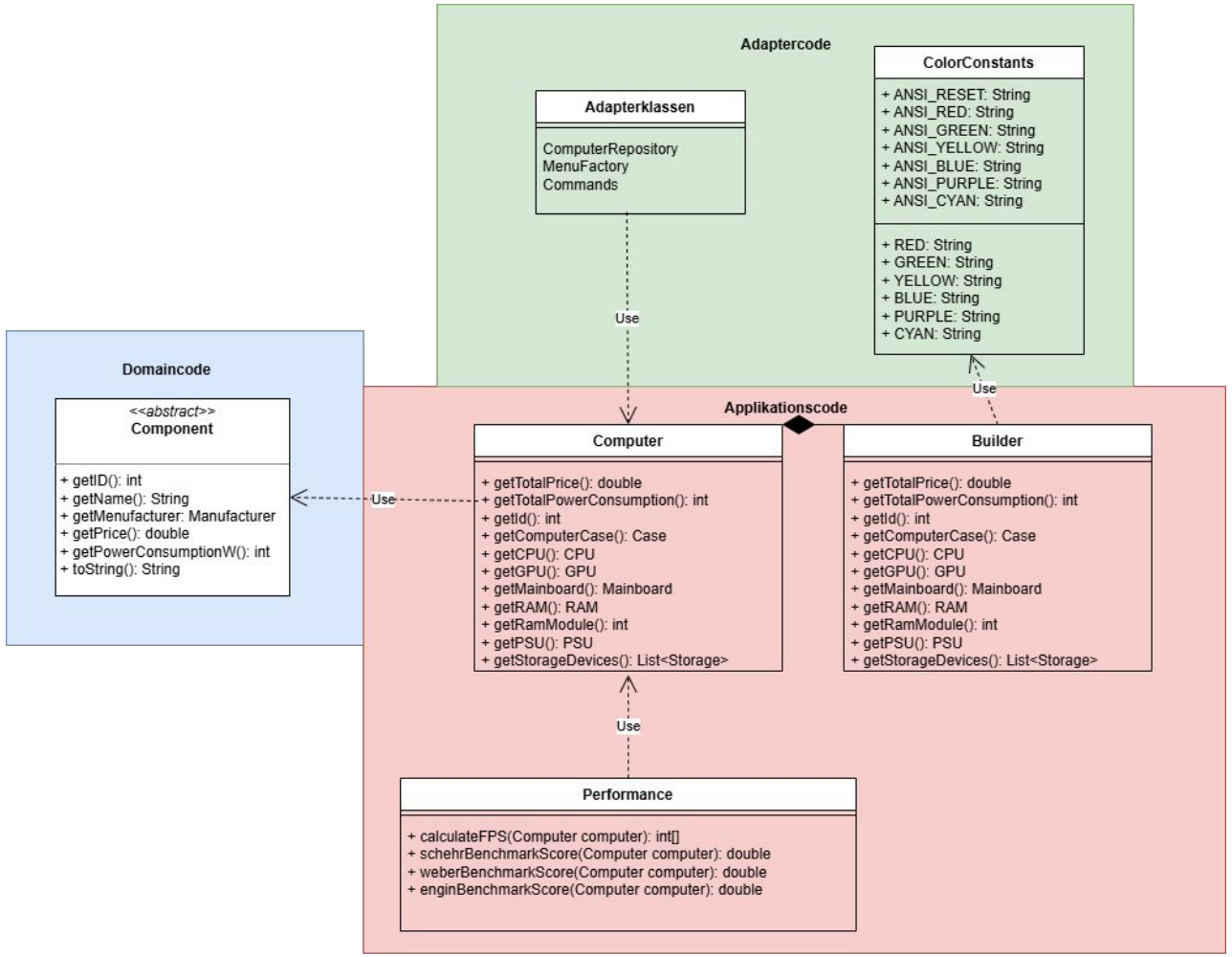
2.2 Dependency Rule - Negativ

Computer.Builder.java

```java
package de.ase.pcpartpicker.part_assembly;

import java.util.ArrayList;
import java.util.List;

import de.ase.pcpartpicker.ColorConstants;
import de.ase.pcpartpicker.domain.CPU;
import de.ase.pcpartpicker.domain.Case;
import de.ase.pcpartpicker.domain.Component;
import de.ase.pcpartpicker.domain.GPU;
import de.ase.pcpartpicker.domain.Mainboard;
import de.ase.pcpartpicker.domain.PSU;
import de.ase.pcpartpicker.domain.RAM;
import de.ase.pcpartpicker.domain.Storage;

/**
 * Klasse die einen vollständig konfigurierten Computer abbildet
 * Diese Klasse ist unveränderlich. <br>
 * {@link Computer.Builder} dient zum Erstellen von Computer-Objekten
 *
 * @author Tuluhan
 */
public class Computer {

    private final int id;
    private final Case computerCase;
    private final CPU cpu;
    private final GPU gpu;
    private final Mainboard mainboard;
    private final RAM ram;
    private final int ramModule; // Anzahl der RAM-Module, z.B. 2 für 2x8GB
    private final PSU psu;
    private final List<Storage> storageDevices; // Array von Speichermedien (z.B. SSDs, HDDs)

    /**
     * Erzeugt einen Computer basierend auf den Einstellungen im Builder.
     * @param builder Der Builder, der die Komponenten enthält.
     */
    private Computer(Builder builder) {
        this.id = builder.id;
        this.computerCase = builder.computerCase;
        this.cpu = builder.cpu;
        this.gpu = builder.gpu;
        this.mainboard = builder.mainboard;
        this.ram = builder.ram;
        this.ramModule = builder.ramModule;
        this.psu = builder.psu;
        this.storageDevices = builder.storageDevices;
    }

```

```java
/**
         * Diese Methode überprüft die Kompatibilität der Komponenten.
         * Es wird lediglich überprüft, ob der PC so gebaut werden und existieren könnte.
         * Bspw. ob das Netzteil genug Leistung hat, um die Komponenten zu versorgen, wird hier nicht überprüft.
         * @return true, wenn die Komponenten kompatibel sind und der Computer erstellt werden kann, andernfalls false. Es werden auch Fehlermeldungen ausgegeben, die erklären, warum der Computer nicht erstellt werden kann.
         * @author Tuluhan
         */
        private boolean validate() {

            if(ram == null) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss mindestens ein RAM-Modul ausgewählt werden.");
                return false;
            }

            if(cpu == null) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss eine CPU ausgewählt werden.");
                return false;
            }

            if(!cpu.hasIntegratedGraphics() && gpu == null) {
                System.out.println(ColorConstants.YELLOW("WARNUNG") + " | Der Computer hat keine dedizierte Grafikkarte und die CPU verfügt nicht über integrierte Grafik. Es könnte zu Problemen bei der Anzeige kommen.");
            }

            if(mainboard == null) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Mainboard ausgewählt werden.");
                return false;
            }

            if(psu == null) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Netzteil ausgewählt werden.");
                return false;
            }

            if(computerCase == null) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Gehäuse ausgewählt werden.");
                return false;
            }

            if (!computerCase.getPSUFormFactor().equals(psu.getFormFactor())) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Die Form des Netzteils passt nicht zum Gehäuse.");
                return false;
            }

            if (!checkMainboardCaseCompatibility(mainboard, computerCase)) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Das Mainboard passt nicht ins Gehäuse.");
                return false;
            }

            if(!cpu.getSocket().equals(mainboard.getSocket())) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Die CPU ist nicht mit dem Mainboard kompatibel.");
                return false;
            }

            if(ramModule > mainboard.getRamSlots()) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Das Mainboard hat nicht genug RAM-Slots für die Anzahl der RAM-Module.");
                return false;
            }

            if(storageDevices.isEmpty()) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Es muss mindestens ein Speichermedium ausgewählt werden.");
                return false;
            }

            if(psu.getWattage() < calculateMomentaryPowerConsumption()) {
                System.out.println(ColorConstants.RED("FEHLER") + " | Das Netzteil hat nicht genug Leistung um das System zu versorgen. Das Netzteil muss mindestens " + calculateMomentaryPowerConsumption() + "W liefern, um alle Komponenten zu versorgen.");
                return false;
            }

            return true;
        }
```

2.2 Dependency Rule- Negativ- UML

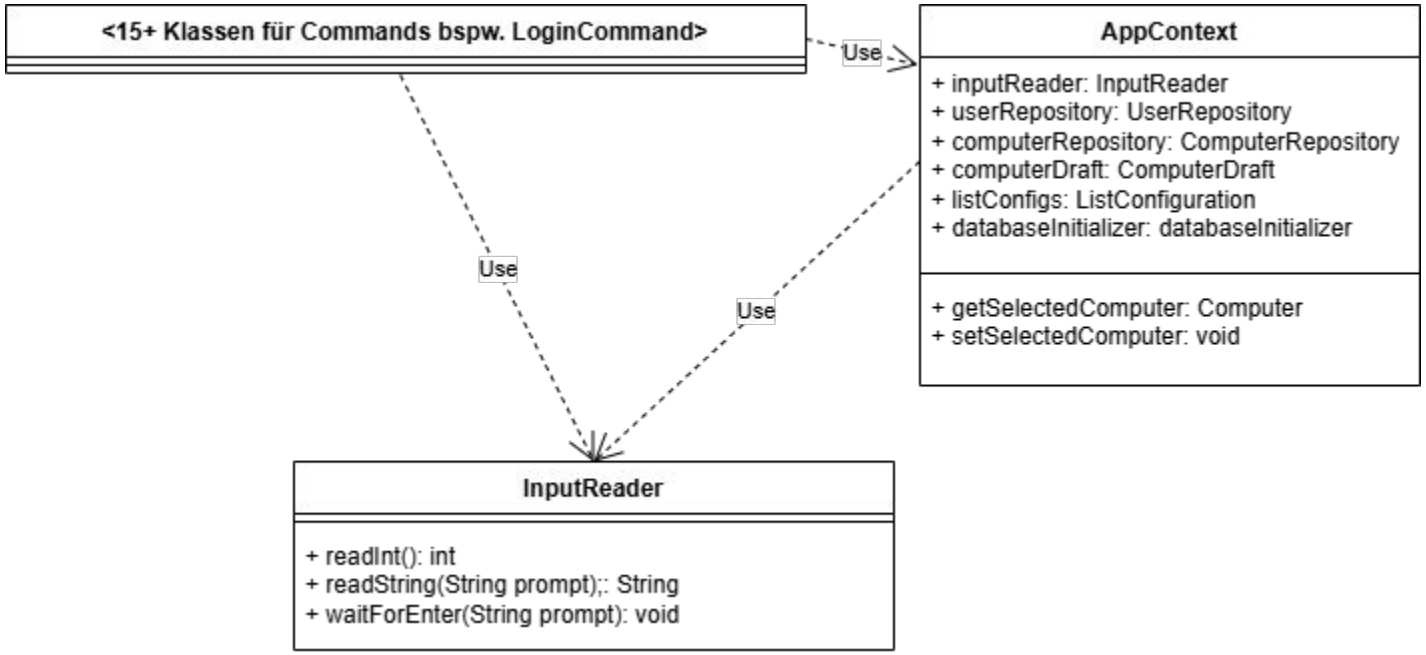


3. SOLID Prinzipien - SRP - Positiv

InputReader.java

```java
package de.ase.pcpartpicker.adapters.cli;

import java.util.Scanner;

/**
 * Klasse, die Tastatureingaben je nach Datentyp liest
 * @author Henri
 */

public class InputReader {
    private final Scanner scanner;

    public InputReader() {
        this.scanner = new Scanner(System.in, "UTF-8");
    }


    public int readInt(String prompt, int min, int max) {
        int input;
        while(true){
            System.out.print(prompt + " (" + min + "-" + max + "): ");
            try{
                input = Integer.parseInt(scanner.nextLine());
                if (input >= min && input <= max) return input;
                System.out.println("Fehler: Bitte eine Zahl zwischen " + min +" und " + max + " eingeben.");
            }
            catch (NumberFormatException e){
                System.out.println("Fehler: Ungültige Eingabe. Bitte eine Zahl eingeben.");
            }
        }
    }


    public String readString(String prompt) {
        System.out.println(prompt + ": ");
        return scanner.nextLine();
    }

    public void waitForEnter(String prompt) {
        System.out.println(prompt);
        scanner.nextLine();
    }
}
```

3. SRP – Positiv – UML



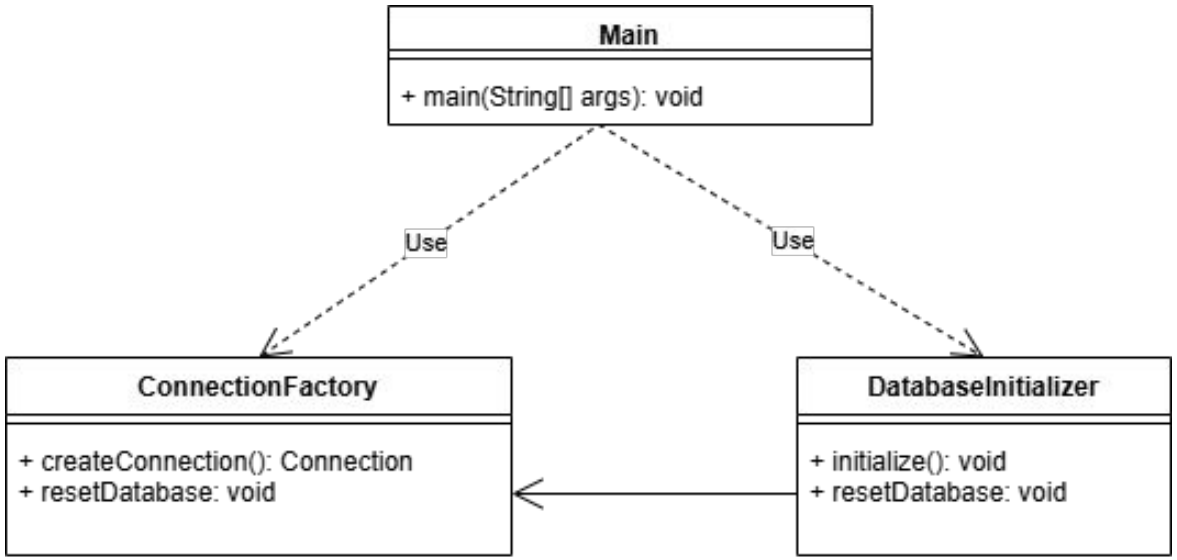
3. SRP – Negativ

DatabaseInitializer.java

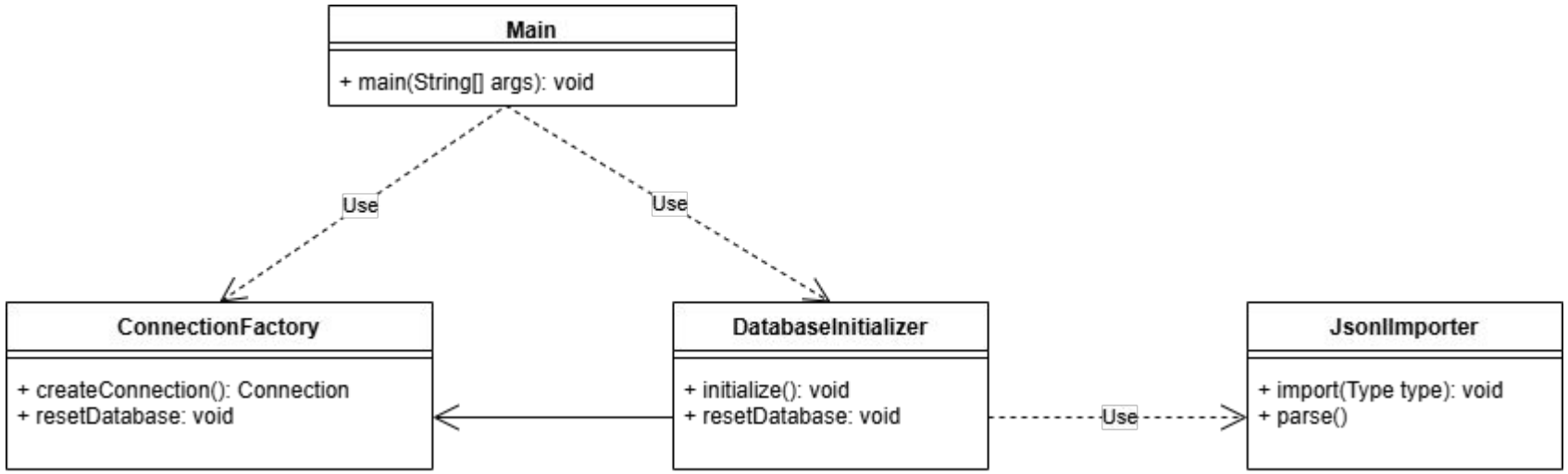
```java
package de.ase.pcpartpicker.adapters.sqlite;

import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Types;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * Klasse zur Initialisierung der SQLite-Datenbank.
 * Erstellt die notwendigen Tabellen, falls sie nicht bereits existieren.
 * @see DatabaseInitializer#initialize()
 * @author Fabio
 */
public class DatabaseInitializer {

    private final ConnectionFactory connectionFactory;
    private final Path dataDirectory;

    public DatabaseInitializer(ConnectionFactory connectionFactory) {
        this(connectionFactory, null);
    }

    public DatabaseInitializer(ConnectionFactory connectionFactory, Path dataDirectory) {
        this.connectionFactory = connectionFactory;
        this.dataDirectory = dataDirectory;
    }

    /**
     * Initialisiert die Datenbank, indem sie die erforderlichen Tabellen erstellt, falls diese nicht bereits existieren.
     * @throws IllegalStateException, wenn die Initialisierung fehlschlägt.
     */
    public void initialize() {
        try (Connection connection = connectionFactory.createConnection();
             Statement statement = connection.createStatement()) {
            statement.execute("PRAGMA foreign_keys = ON");

            createSchema(statement);

            if (areAllComponentTablesEmpty(connection)) {
                seedFromJsonl(connection);
            }
        } catch (SQLException e) {
            throw new IllegalStateException("SQLite-Initialisierung fehlgeschlagen.", e);
        }
    }

```

```java
private void seedFromJsonl(Connection connection) {
        try {
            int cpuTypeId = ensureNamedId(connection, "type", "CPU");
            int gpuTypeId = ensureNamedId(connection, "type", "GPU");
            int ramTypeId = ensureNamedId(connection, "type", "RAM");
            int mainboardTypeId = ensureNamedId(connection, "type", "Mainboard");
            int psuTypeId = ensureNamedId(connection, "type", "PSU");
            int caseTypeId = ensureNamedId(connection, "type", "Case");
            int hddTypeId = ensureNamedId(connection, "type", "HDD");
            int ssdTypeId = ensureNamedId(connection, "type", "SSD");
            int m2SsdTypeId = ensureNamedId(connection, "type", "M2SSD");

            System.out.println("Erstelle CPUs ...");
            importCpu(connection, cpuTypeId);
            System.out.println("Erstelle GPUs ...");
            importGpu(connection, gpuTypeId);
            System.out.println("Erstelle RAM ...");
            importRam(connection, ramTypeId);
            System.out.println("Erstelle Mainboards ...");
            importMainboard(connection, mainboardTypeId);
            System.out.println("Erstelle PSUs ...");
            importPsu(connection, psuTypeId);
            System.out.println("Erstelle Cases ...");
            importCase(connection, caseTypeId);
            System.out.println("Erstelle Speicher ...");
            importStorage(connection, hddTypeId, ssdTypeId, m2SsdTypeId);
            System.out.println("Import abgeschlossen.");
        } catch (SQLException e) {
            throw new IllegalStateException("Seed aus JSONL fehlgeschlagen.", e);
        }
    }

    private void importCpu(Connection connection, int typeId) throws SQLException {
        String sql = """
            INSERT INTO cpu (manufacturer_id, type_id, name, price, socket_id, speed_ghz, boost_clock, hasIntegratedGraphics, power_consumption_w)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
            """;

        importRows(connection, "cpu.jsonl", sql, (row, ps) -> {
            String name = getString(row, "name");
            Double price = getDouble(row, "price");
            Double speed = getDouble(row, "core_clock");
            Double boost = getDouble(row, "boost_clock");
            Integer tdp = getInt(row, "tdp");
            if (anyNull(name, price, speed, tdp)) {
                return false;
            }

            String manufacturer = extractManufacturer(name);
            String cleanName = stripManufacturerPrefix(name);
            String socket = inferCpuSocket(name, getString(row, "microarchitecture"));
            boolean hasIgpu = !isJsonNullOrMissing(row, "graphics");

            int manufacturerId = ensureNamedId(connection, "manufacturer", manufacturer);
            int socketId = ensureNamedId(connection, "sockets", socket);

            ps.setInt(1, manufacturerId);
            ps.setInt(2, typeId);
            ps.setString(3, cleanName);
            ps.setDouble(4, price);
            ps.setInt(5, socketId);
            ps.setDouble(6, speed);
            if (boost == null) {
                ps.setNull(7, Types.REAL);
            } else {
                ps.setDouble(7, boost);
            }
            ps.setBoolean(8, hasIgpu);
            ps.setInt(9, tdp);
            return true;
        });
    }
```

3. SRP – Negativ - UML



3. SRP – Negativ – UML - Lösung



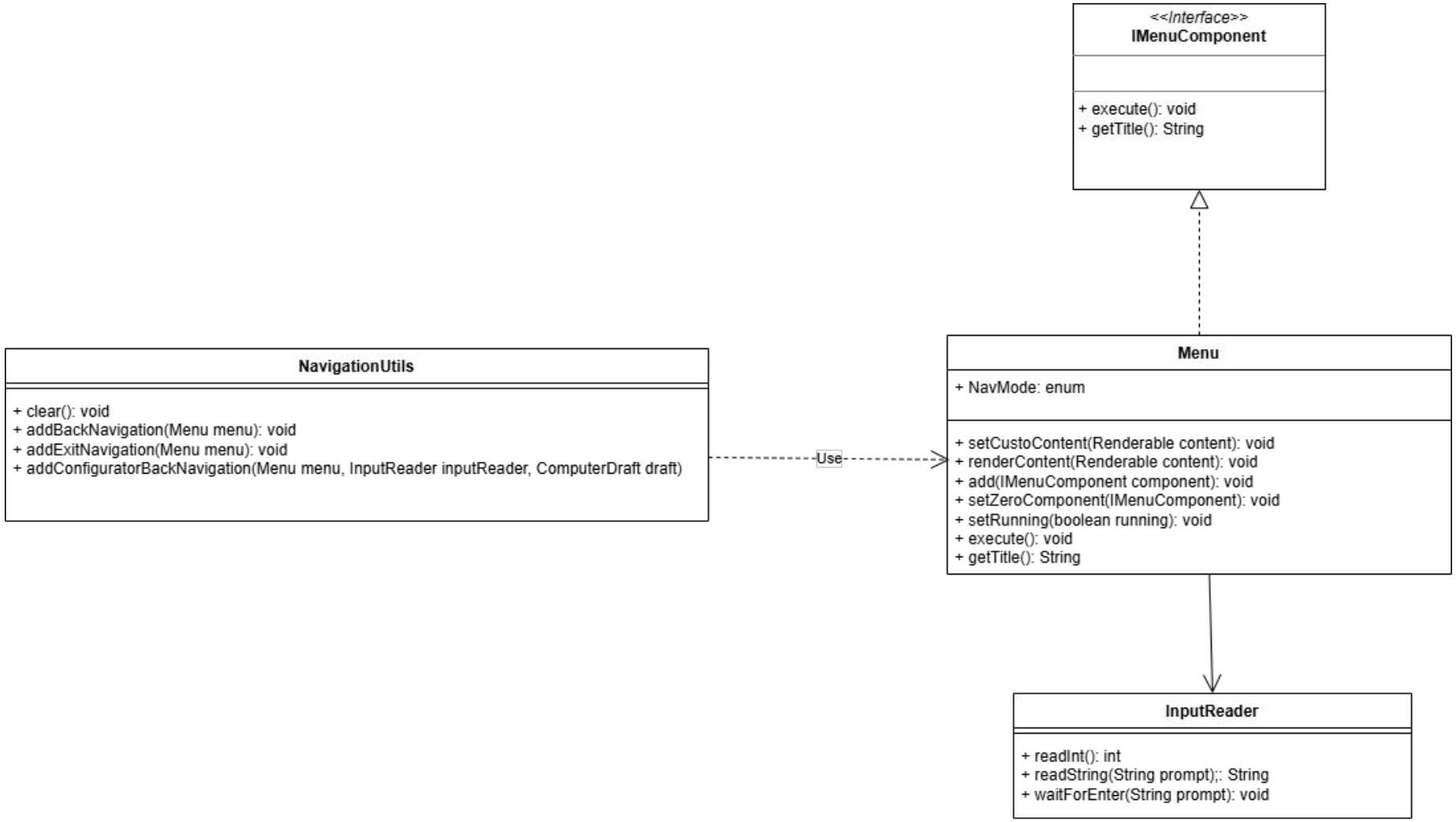
3.1 Open/Close Principle – Positiv

Menu.java

```java
package de.ase.pcpartpicker.adapters.cli;

import java.util.ArrayList;
import java.util.List;

import de.ase.pcpartpicker.adapters.cli.utils.NavigationUtils;
/**
 * Klasse, die den grundsätzlichen Aufbau eines Menüs definiert
 * @param title Titel des Menüs
 * @param children Enthält Untermenüs, die im aktuellen Menü angezeigt werden können
 * @param inputReader liest die Benutzereingaben von der Konsole
 * @param running Flag das angibt, ob das Menü gerade aktiv ist
 * @author Henri
*/
public class Menu implements IMenuComponent {
    public enum NavMode {STANDARD, PAGING};

    private final String title;
    private final List<IMenuComponent> children = new ArrayList<>();
    private IMenuComponent zeroComponent;
    private final InputReader inputReader;
    private boolean running = true;
    private Renderable customContent;
    private final NavMode nav;



    public Menu(String title, InputReader inputReader, NavMode nav) {
        this.title = title;
        this.inputReader = inputReader;
        this.nav = nav;
    }

    public Menu(String title, InputReader inputReader) {
        // setze den Standard naviagtionsmodus
        this(title, inputReader, NavMode.STANDARD);
    }


    /**
     * Setzt beliebigen renderbaren Inhalt, der im Menü angezeigt werden soll
     */
    public void setCustomContent(Renderable content) {
        this.customContent = content;
    }
```

```java
    /**
     * Gibt beliebigen renderbaren Inhalt direkt im Menü aus
     */
    public void renderContent(Renderable content) {
        if (content != null) {
            content.render(title);
        }
    }

    public void add(IMenuComponent component) {
        children.add(component);
    }

    public void setZeroComponent(IMenuComponent component) {
        this.zeroComponent = component;
    }

    public void setRunning(boolean running){
        this.running = running;
    }

    @Override
    public void execute() {
        running = true;
        while (running) {


            if(nav == NavMode.PAGING && customContent != null) {
                customContent.render(title);
                running=false;
                return;
            }

            NavigationUtils.clear();
            System.out.println("\n=== " + title + " ===\n");

            if(SessionManager.isLoggedIn()) {
                System.out.println("Eingeloggt als: "+ SessionManager.getcurrentUser().getName()+ "\n");
            }

            if(customContent != null) {
                customContent.render(title);
            }

            for (int i = 0; i < children.size(); i++) {
                System.out.println((i + 1) + ") " + children.get(i).getTitle());
            }

            if (zeroComponent != null) {
                System.out.println("0) " + zeroComponent.getTitle());
            }


            System.out.print("\nAuswahl: ");

            int choice = inputReader.readInt("Eine Zahl eingeben", 0, children.size());


            if (choice == 0 && zeroComponent != null) {
                zeroComponent.execute();
            } else if (choice > 0 && choice <= children.size()) {
                children.get(choice - 1).execute();
            }

        }
    }
    @Override
    public String getTitle() {
        return title;
    }

}
```

3.1 Open/Close Principle – Positiv - UML



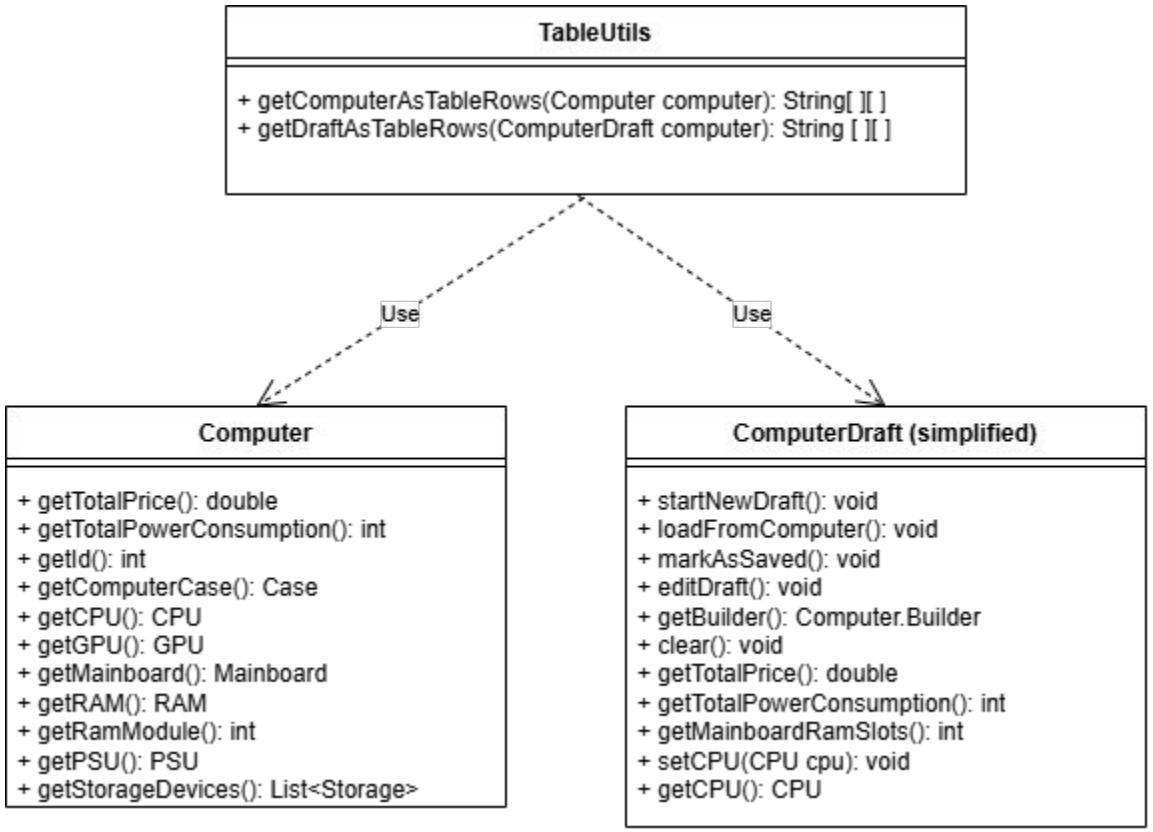
3.1 Open/Close Principle – Negativ

TableUtils.java

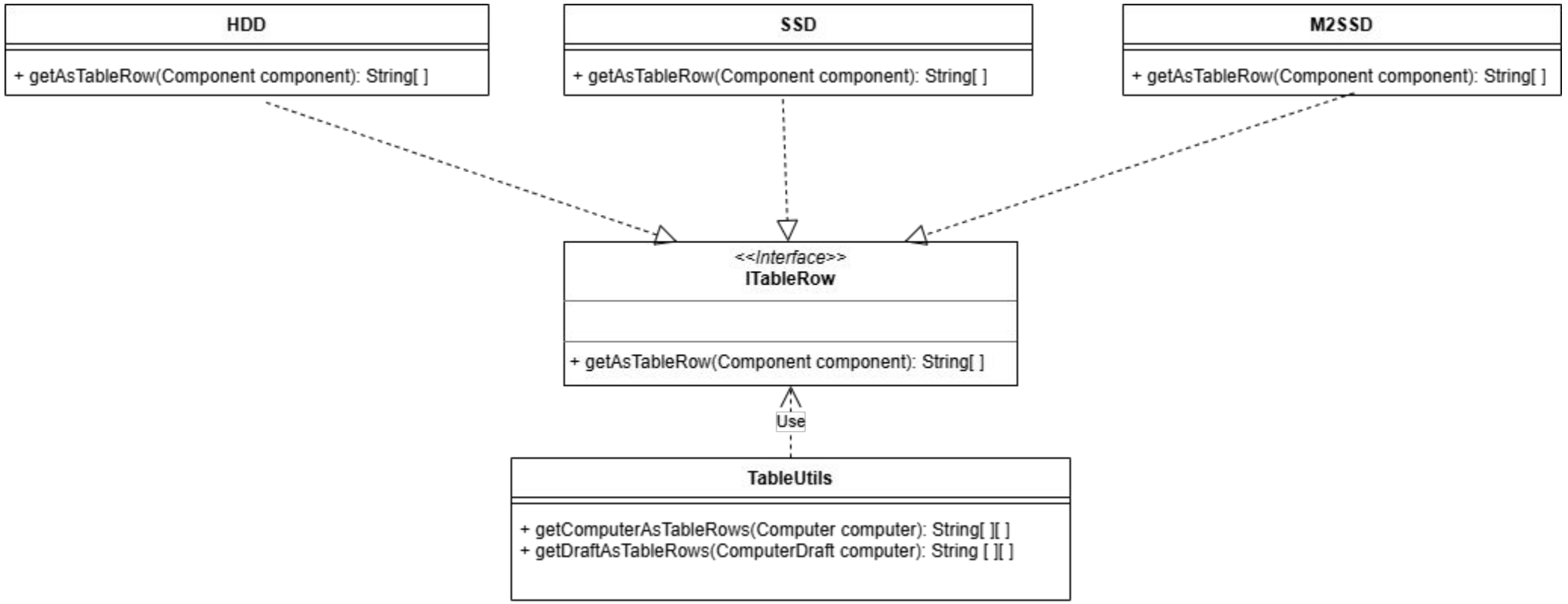
```java
package de.ase.pcpartpicker.adapters.cli.utils;

import java.util.List;

import de.ase.pcpartpicker.adapters.cli.ComputerDraft;
import de.ase.pcpartpicker.domain.CPU;
import de.ase.pcpartpicker.domain.Case;
import de.ase.pcpartpicker.domain.GPU;
import de.ase.pcpartpicker.domain.Mainboard;
import de.ase.pcpartpicker.domain.PSU;
import de.ase.pcpartpicker.domain.RAM;
import de.ase.pcpartpicker.domain.Storage;
import de.ase.pcpartpicker.part_assembly.Computer;

public class TableUtils {

    public static String[][] getComputerAsTableRows(Computer computer) {
        return buildTableRows(
            computer.getComputerCase(), computer.getCPU(), computer.getGPU(),
            computer.getMainboard(), computer.getRAM(), computer.getRamModule(),
            computer.getPSU(), computer.getStorageDevices(),
            computer.getTotalPowerConsumption(), computer.getTotalPrice()
        );
    }

    // 2. Methode für den unfertigen ComputerDraft
    public static String[][] getDraftAsTableRows(ComputerDraft draft) {
        return buildTableRows(
            draft.getComputerCase(), draft.getCPU(), draft.getGPU(),
            draft.getMainboard(), draft.getRAM(), draft.getRamModule(),
            draft.getPSU(), draft.getStorage(),
            draft.getTotalPowerConsumption(), draft.getTotalPrice()
        );
    }

    private static String[][] buildTableRows(Case pcCase, CPU cpu, GPU gpu, Mainboard mb, RAM ram, int ramModule, PSU psu, List<Storage> storage, int totalPower, double totalPrice) {
        List<String[]> rows = new java.util.ArrayList<>();

        // Gehäuse
        if (pcCase != null) {
            rows.add(new String[]{"Gehäuse", "Name", pcCase.getName()});
            rows.add(new String[]{"", "Formfaktor", pcCase.getMotherboardFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(pcCase.getPrice())});
        } else {
            rows.add(new String[]{"Gehäuse", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // CPU
        if (cpu != null) {
            rows.add(new String[]{"CPU", "Name", cpu.getName()});
            rows.add(new String[]{"", "Kerne", FormatUtils.formatNumber(cpu.getCoreCount())});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(cpu.getSpeedGHz()) + " GHz"}); // Tippfehler bei dir korrigiert (formatNumer -> formatNumber)
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(cpu.getPrice())});
        } else {
            rows.add(new String[]{"CPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // GPU
        if (gpu != null) {
            rows.add(new String[]{"GPU", "Name", gpu.getName()});
            rows.add(new String[]{"", "VRAM", FormatUtils.formatNumber(gpu.getVramGB()) + " GB"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(gpu.getPrice())});
        } else {
            rows.add(new String[]{"GPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Mainboard
        if (mb != null) {
            rows.add(new String[]{"Mainboard", "Name", mb.getName()});
            rows.add(new String[]{"", "Formfaktor", mb.getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(mb.getPrice())});
        } else {
            rows.add(new String[]{"Mainboard", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});


```

```java

        // RAM
        if (ram != null) {
            rows.add(new String[]{"RAM", "Name", ram.getName()});
            rows.add(new String[]{"", "Module", FormatUtils.formatNumber(ramModule)});
            rows.add(new String[]{"", "Gesamtkapazität", FormatUtils.formatNumber(getRAMCapacity(ramModule, ram.getCapacityGB())) + " GB"});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(ram.getSpeedMHz()) + " MHz"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(ramModule * ram.getPrice())});
        } else {
            rows.add(new String[]{"RAM", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Netzteil
        if (psu != null) {
            rows.add(new String[]{"Netzteil", "Name", psu.getName()});
            rows.add(new String[]{"", "Leistung", FormatUtils.formatNumber(psu.getWattage()) + " W"});
            rows.add(new String[]{"", "Formfaktor", psu.getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(psu.getPrice())});
        } else {
            rows.add(new String[]{"Netzteil", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Storage
        if (storage != null && !storage.isEmpty()) {
            int i = 0;
            for (Storage s : storage) {
                String comp = (i == 0) ? "Speicher" : "";
                rows.add(new String[]{comp, "Name", s.getName()});
                rows.add(new String[]{"", "Kapazität", FormatUtils.formatNumber(s.getCapacityGB()) + " GB"});
                rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(s.getPrice())});
                rows.add(new String[]{"", "", ""});
                i++;
            }
        } else {
            rows.add(new String[]{"Speicher", "nicht ausgewählt", ""});
            rows.add(new String[]{"", "", ""});
        }

        // Trennlinie und Gesamtpreis (Logik vereint)
        rows.add(new String[]{"---", "---", "---"});
        String psuWattage = psu != null ? FormatUtils.formatNumber(psu.getWattage()) + " W" : "0 W";
        rows.add(new String[]{"Leistungsaufnahme/Gesamt", "", FormatUtils.formatNumber(totalPower) + " W / " + psuWattage});
        rows.add(new String[]{"Gesamtpreis", "", FormatUtils.formatPrice(totalPrice)});

        return rows.toArray(new String[0][]);
    }

    private static int getRAMCapacity(int module, int capacity) {
        return module * capacity;
    }
}

```

3.1 Open/Close Principle – Negativ - UML



3.1 Open/Close Principle – Negativ – UML- Lösung



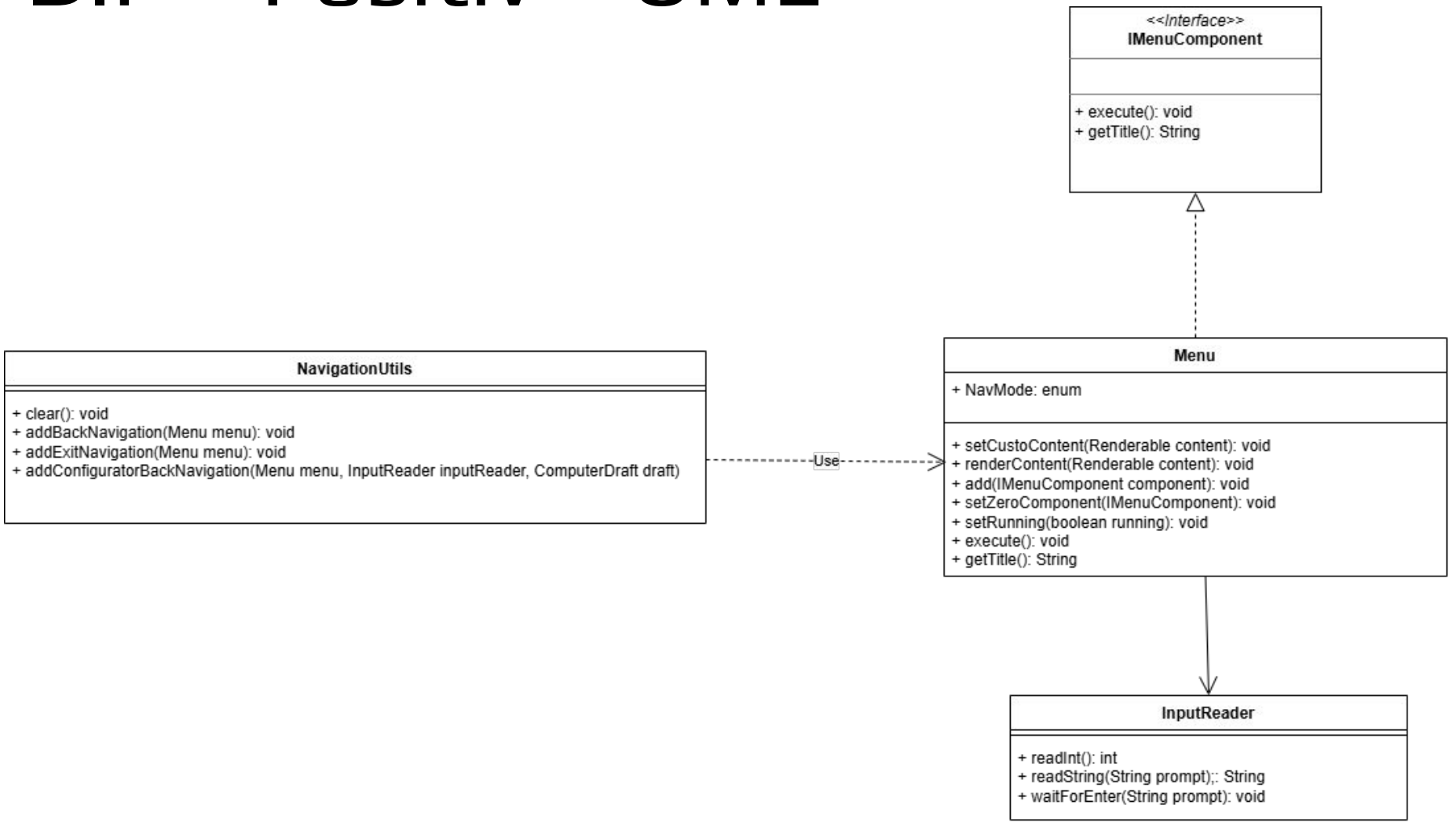
3.2 Dependency Inversion Principle (DIP) - Positiv

Menu.java

```java
package de.ase.pcpartpicker.adapters.cli;

import java.util.ArrayList;
import java.util.List;

import de.ase.pcpartpicker.adapters.cli.utils.NavigationUtils;
/**
 * Klasse, die den grundsätzlichen Aufbau eines Menüs definiert
 * @param title Titel des Menüs
 * @param children Enthält Untermenüs, die im aktuellen Menü angezeigt werden können
 * @param inputReader liest die Benutzereingaben von der Konsole
 * @param running Flag das angibt, ob das Menü gerade aktiv ist
 * @author Henri
 */
public class Menu implements IMenuComponent {
    public enum NavMode {STANDARD, PAGING};

    private final String title;
    private final List<IMenuComponent> children = new ArrayList<>();
    private IMenuComponent zeroComponent;
    private final InputReader inputReader;
    private boolean running = true;
    private Renderable customContent;
    private final NavMode nav;



    public Menu(String title, InputReader inputReader, NavMode nav) {
        this.title = title;
        this.inputReader = inputReader;
        this.nav = nav;
    }

    public Menu(String title, InputReader inputReader) {
        // setze den Standard naviagtionsmodus
        this(title, inputReader, NavMode.STANDARD);
    }


    /**
     * Setzt beliebigen renderbaren Inhalt, der im Menü angezeigt werden soll
     */
    public void setCustomContent(Renderable content) {
        this.customContent = content;
    }
```

```java
    /**
     * Gibt beliebigen renderbaren Inhalt direkt im Menü aus
     */
    public void renderContent(Renderable content) {
        if (content != null) {
            content.render(title);
        }
    }

    public void add(IMenuComponent component) {
        children.add(component);
    }

    public void setZeroComponent(IMenuComponent component) {
        this.zeroComponent = component;
    }

    public void setRunning(boolean running){
        this.running = running;
    }

    @Override
    public void execute() {
        running = true;
        while (running) {


            if(nav == NavMode.PAGING && customContent != null) {
                customContent.render(title);
                running=false;
                return;
            }

            NavigationUtils.clear();
            System.out.println("\n=== " + title + " ===\n");

            if(SessionManager.isLoggedIn()) {
                System.out.println("Eingeloggt als: "+ SessionManager.getcurrentUser().getName()+ "\n");
            }

            if(customContent != null) {
                customContent.render(title);
            }

            for (int i = 0; i < children.size(); i++) {
                System.out.println((i + 1) + ") " + children.get(i).getTitle());
            }

            if (zeroComponent != null) {
                System.out.println("0) " + zeroComponent.getTitle());
            }


            System.out.print("\nAuswahl: ");

            int choice = inputReader.readInt("Eine Zahl eingeben", 0, children.size());


            if (choice == 0 && zeroComponent != null) {
                zeroComponent.execute();
            } else if (choice > 0 && choice <= children.size()) {
                children.get(choice - 1).execute();
            }

        }
    }

    @Override
    public String getTitle() {
        return title;
    }

}
```

3.2 DIP – Positiv – UML

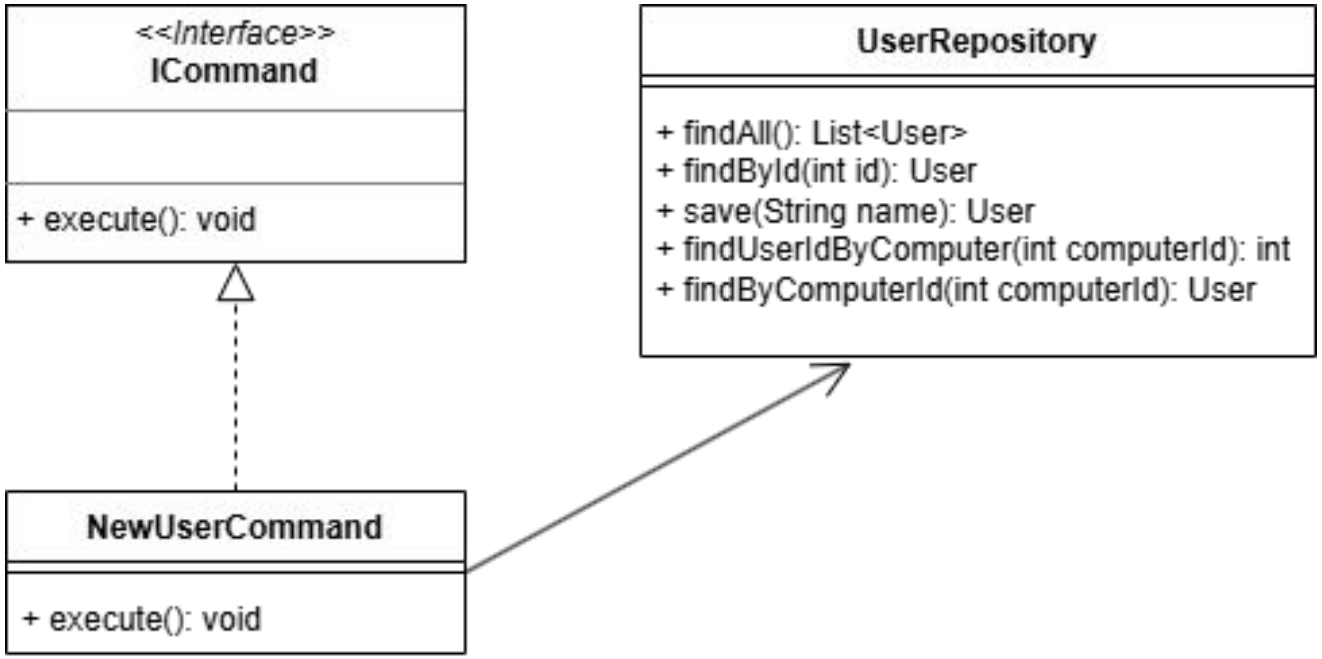


3.2 DIP- Negativ

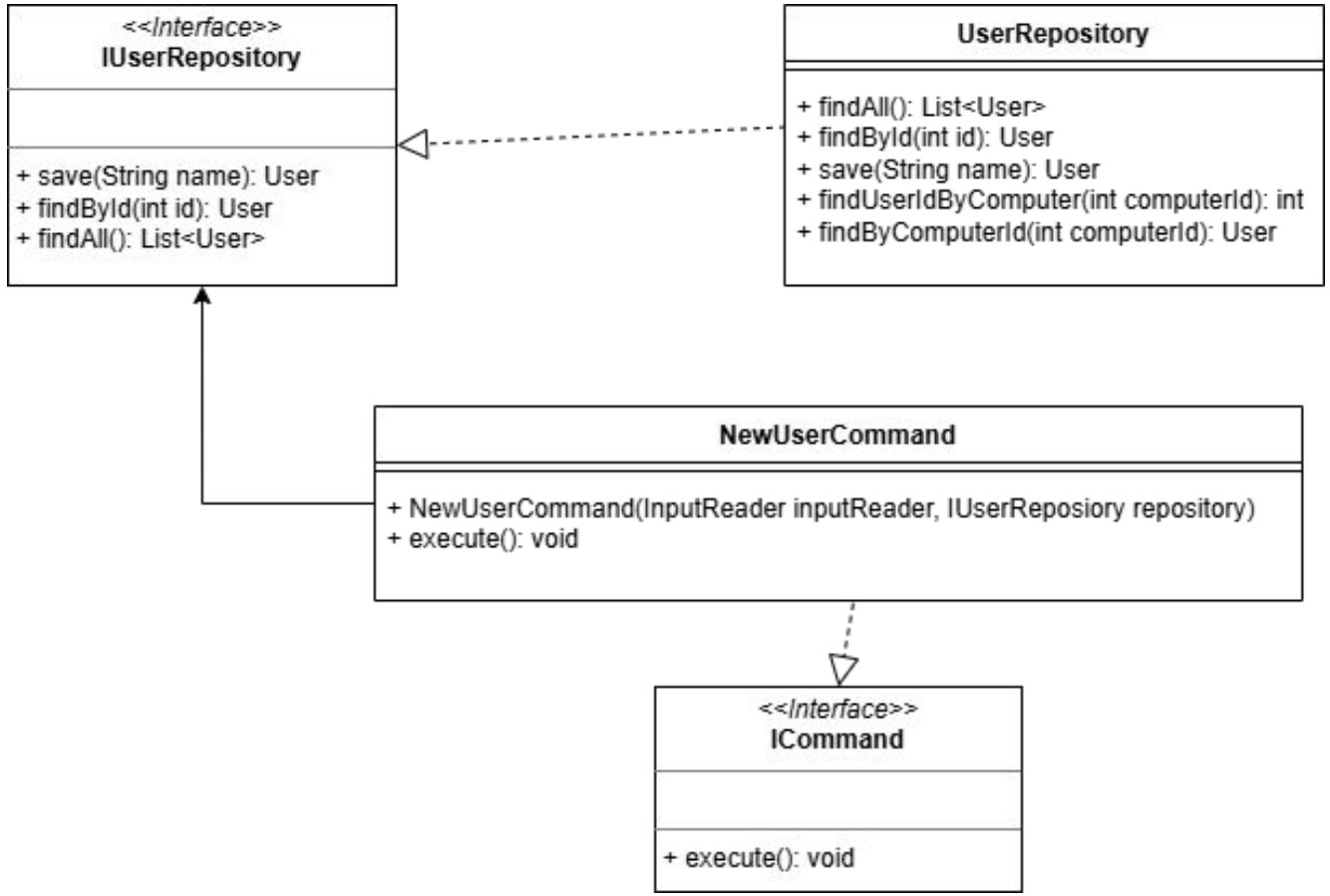
NewUserCommand.java

```java
package de.ase.pcpartpicker.adapters.cli.commands;

import de.ase.pcpartpicker.adapters.cli.InputReader;
import de.ase.pcpartpicker.adapters.cli.utils.ExceptionUtils;
import de.ase.pcpartpicker.adapters.sqlite.repositories.UserRepository;


public class NewUserCommand implements ICommand{

    private final InputReader inputReader;
    private final UserRepository userRepository;

    public NewUserCommand(InputReader inputReader, UserRepository userRepository) {
        this.inputReader = inputReader;
        this.userRepository = userRepository;
    }

    @Override
    public void execute() {
        String username = inputReader.readString("Bitte Nutzernamen eingeben");
        boolean exists = userRepository.findAll().stream()
            .anyMatch(user -> user.getName().equalsIgnoreCase(username));

        if (exists) {
            ExceptionUtils.printInfo("Nutzername existiert bereits!");
        } else {
            userRepository.save(username);
            ExceptionUtils.printInfo("Nutzer erfolgreich angelegt!");
        }

        inputReader.waitForEnter("\nEnter drücken zum fortfahren...");

    }



}

```

3.2 DIP- Negativ - UML



3.2 DIP- Negativ – UML - Lösung



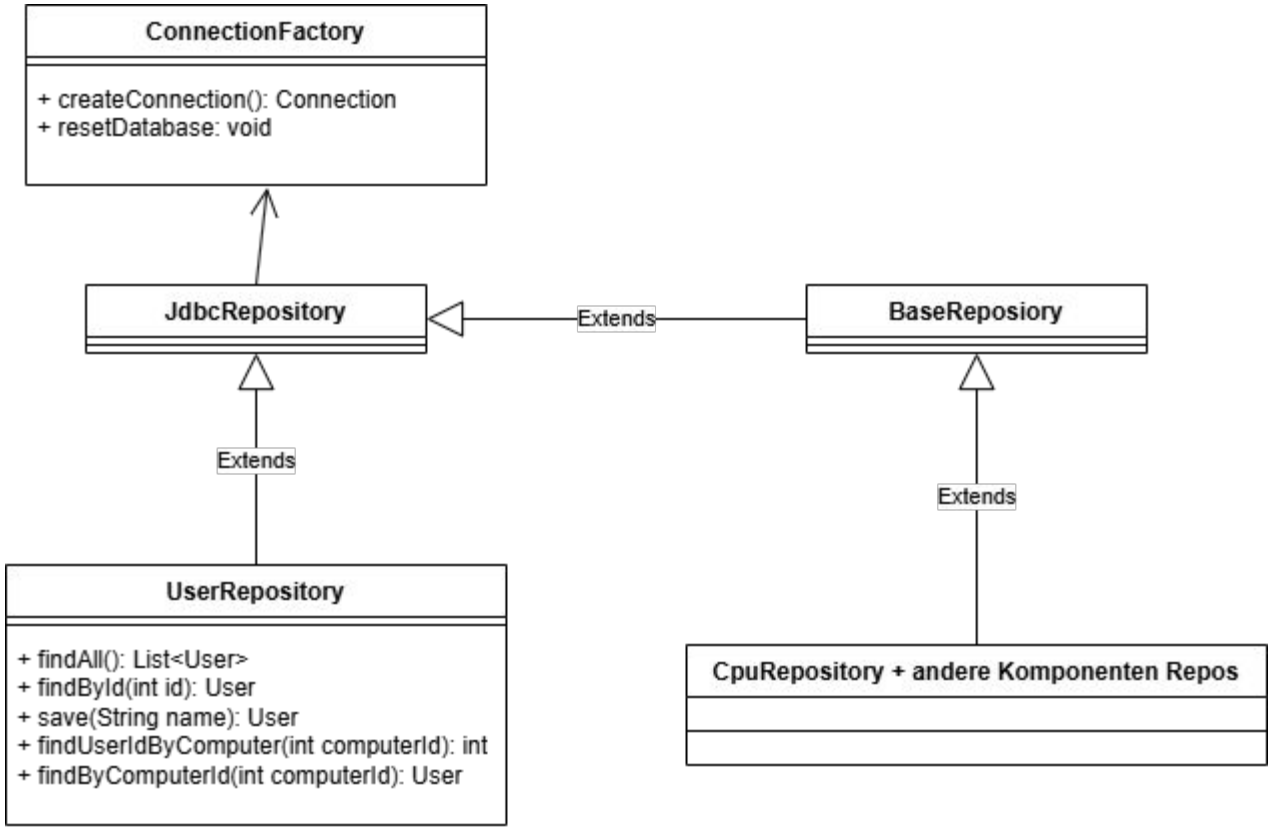
4. GRASP – Low Coupeling - Positiv

JdbcRepository.java

```java
package de.ase.pcpartpicker.adapters.sqlite.repositories;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import de.ase.pcpartpicker.adapters.sqlite.ConnectionFactory;

/**
 * Gemeinsame JDBC-Basisklasse für Repositorys.
 *
 * @param <T> Typ des Domain-Objekts.
 */
public abstract class JdbcRepository<T> implements Repository<T> {

    @FunctionalInterface
    protected interface RowMapper<R> {
        R map(ResultSet resultSet) throws SQLException;
    }

    @FunctionalInterface
    protected interface StatementConfigurer {
        void configure(PreparedStatement statement) throws SQLException;
    }

    protected static final StatementConfigurer NO_PARAMETERS = statement -> { };

    protected final ConnectionFactory connectionFactory;

    protected JdbcRepository(ConnectionFactory connectionFactory) {
        this.connectionFactory = connectionFactory;
    }

    protected <R> List<R> queryList(String sql, RowMapper<R> rowMapper, String errorMessage) {
        return queryList(sql, NO_PARAMETERS, rowMapper, errorMessage);
    }

    protected <R> List<R> queryList(
        String sql,
        StatementConfigurer statementConfigurer,
        RowMapper<R> rowMapper,
        String errorMessage
    ) {
        List<R> results = new ArrayList<>();

        try (Connection connection = connectionFactory.createConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statementConfigurer.configure(statement);

            try (ResultSet resultSet = statement.executeQuery()) {
                while (resultSet.next()) {
                    results.add(rowMapper.map(resultSet));
                }
            }
            return results;

        } catch (SQLException e) {
            throw new IllegalStateException(errorMessage, e);
        }
    }

```

```java
    protected <R> Optional<R> queryOptional(String sql, RowMapper<R> rowMapper, String errorMessage) {
        return queryOptional(sql, NO_PARAMETERS, rowMapper, errorMessage);
    }

    protected <R> Optional<R> queryOptional(
        String sql,
        StatementConfigurer statementConfigurer,
        RowMapper<R> rowMapper,
        String errorMessage
    ) {
        try (Connection connection = connectionFactory.createConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statementConfigurer.configure(statement);

            try (ResultSet resultSet = statement.executeQuery()) {
                if (!resultSet.next()) {
                    return Optional.empty();
                }
                return Optional.of(rowMapper.map(resultSet));
            }

        } catch (SQLException e) {
            throw new IllegalStateException(errorMessage, e);
        }
    }

    protected int insertAndReturnGeneratedKey(
        String sql,
        StatementConfigurer statementConfigurer,
        String errorMessage
    ) {
        try (Connection connection = connectionFactory.createConnection();
             PreparedStatement statement = connection.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

            statementConfigurer.configure(statement);
            statement.executeUpdate();

            try (ResultSet generatedKeys = statement.getGeneratedKeys()) {
                if (generatedKeys.next()) {
                    return generatedKeys.getInt(1);
                }
            }
            throw new IllegalStateException(errorMessage + ": Keine ID zurückgegeben.");

        } catch (SQLException e) {
            throw new IllegalStateException(errorMessage, e);
        }
    }

    protected int executeUpdate(String sql, StatementConfigurer statementConfigurer, String errorMessage) {
        try (Connection connection = connectionFactory.createConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statementConfigurer.configure(statement);
            return statement.executeUpdate();

        } catch (SQLException e) {
            throw new IllegalStateException(errorMessage, e);
        }
    }
}
```

4. Low Coupeling – Positiv – UML

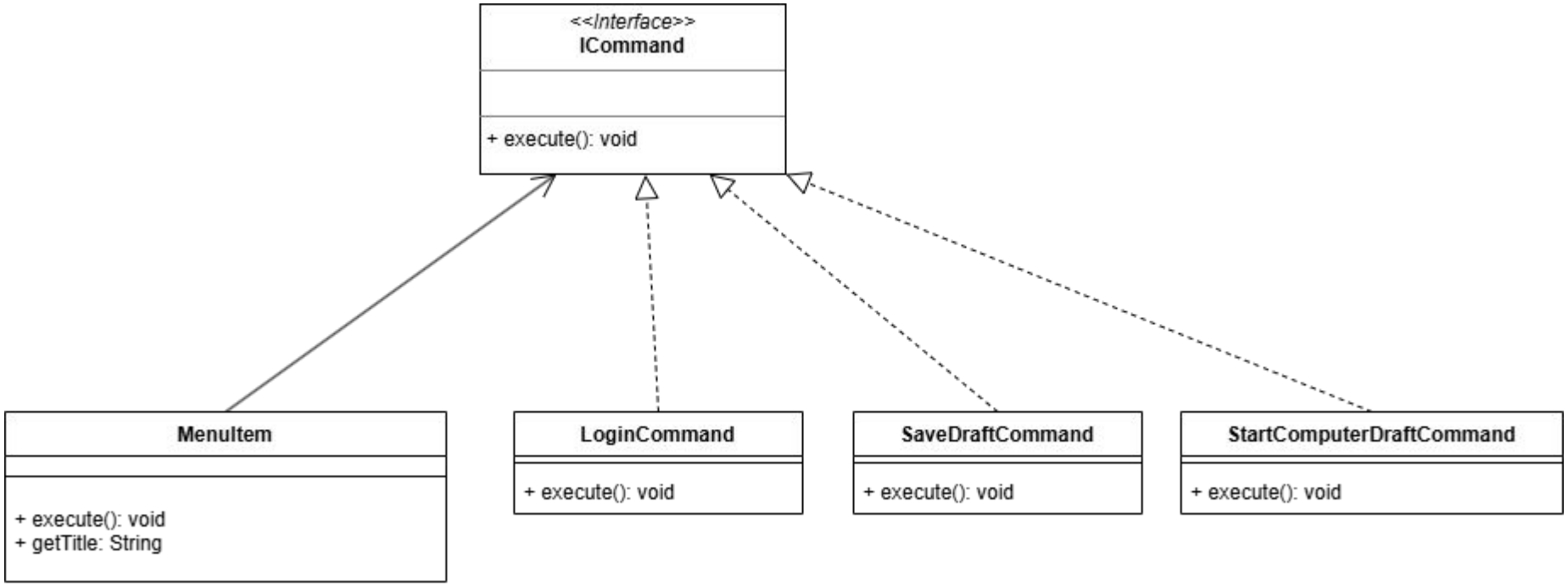


4.1 Polymorphismus - Positiv

MenuItem.java

```java
package de.ase.pcpartpicker.adapters.cli;

import java.util.function.Supplier;

import de.ase.pcpartpicker.adapters.cli.commands.ICommand;
/**
 * Klasse, die definiert welche Komponenten ein Menüeintrag hat
 * @param title Name des Menüeintrages
 * @param command Speichert die Aktion die Ausgeführt werden soll
 * @author Henri
 */
public class MenuItem implements IMenuComponent {
    private final Supplier<String> titleSupplier;
    private final ICommand command;
    private Supplier<Boolean> visibilitySupplier =  () -> true;

    public MenuItem(String title, ICommand command) {
        this.titleSupplier = () -> title;
        this.command = command;
    }

    // Konstruktor für dynamische Titel
    public MenuItem(Supplier<String> titleSupplier, ICommand command) {
        this.titleSupplier = titleSupplier;
        this.command = command;
    }

    public MenuItem(String title, ICommand command, Supplier<Boolean> visibilitySupplier) {
        this.titleSupplier =  () -> title;
        this.command = command;
        this.visibilitySupplier = visibilitySupplier;
    }

    @Override
    public boolean isVisible() {
        return visibilitySupplier.get();
    }
    @Override
    public void execute() {
        command.execute();
    }

    @Override
    public String getTitle() {
        return titleSupplier.get();
    }

}
```

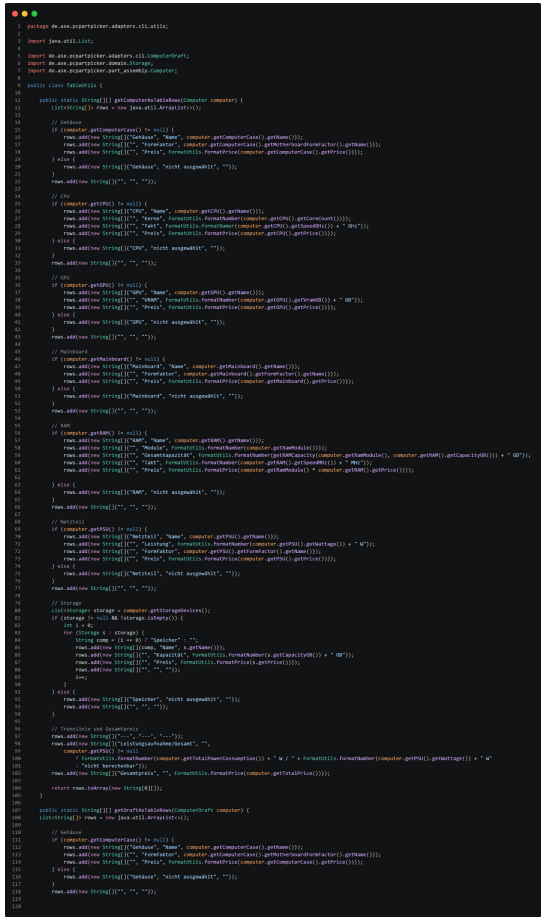
4.1 Polymorphismus - Positiv - UML



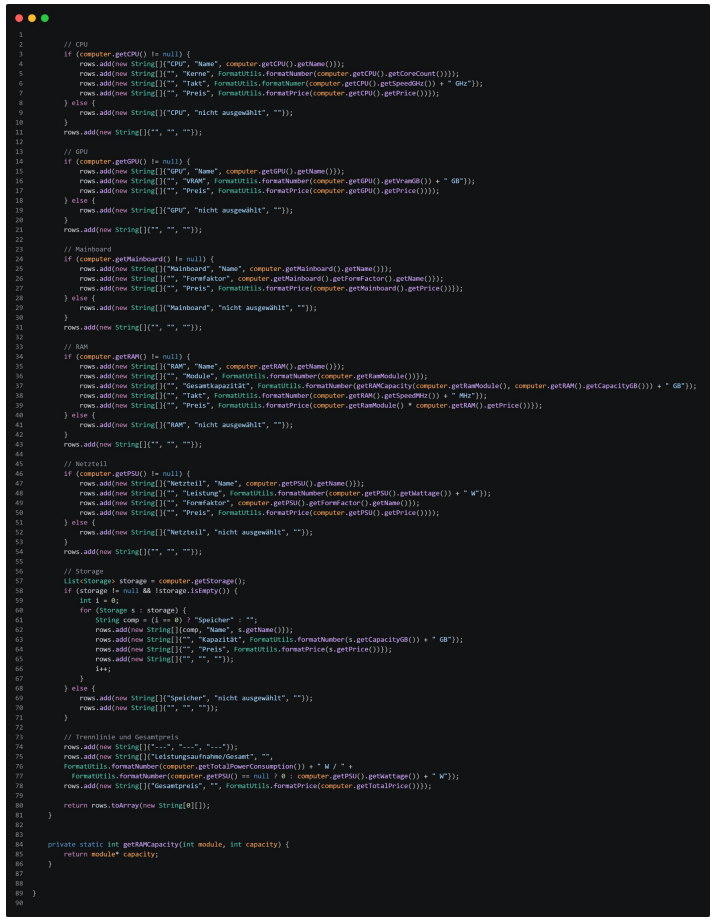
4.2 Dont Repeat Yourself (DRY) - Vorher

TableUtils.java

Commit: 72507d2



```java
package de.ase.pcpartpicker.adapters.cli.utils;

import java.util.List;

import de.ase.pcpartpicker.adapters.cli.ComputerDraft;
import de.ase.pcpartpicker.domain.Storage;
import de.ase.pcpartpicker.part_assembly.Computer;

public class TableUtils {

    public static String[][] getComputerAsTableRows(Computer computer) {
        List<String[]> rows = new java.util.ArrayList<>();

        // Gehäuse
        if (computer.getComputerCase() != null) {
            rows.add(new String[]{"Gehäuse", "Name", computer.getComputerCase().getName()});
            rows.add(new String[]{"", "Formfaktor", computer.getComputerCase().getMotherboardFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getComputerCase().getPrice())});
        } else {
            rows.add(new String[]{"Gehäuse", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // CPU
        if (computer.getCPU() != null) {
            rows.add(new String[]{"CPU", "Name", computer.getCPU().getName()});
            rows.add(new String[]{"", "Kerne", FormatUtils.formatNumber(computer.getCPU().getCoreCount())});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(computer.getCPU().getSpeedGHz()) + " GHz"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getCPU().getPrice())});
        } else {
            rows.add(new String[]{"CPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // GPU
        if (computer.getGPU() != null) {
            rows.add(new String[]{"GPU", "Name", computer.getGPU().getName()});
            rows.add(new String[]{"", "VRAM", FormatUtils.formatNumber(computer.getGPU().getVramGB()) + " GB"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getGPU().getPrice())});
        } else {
            rows.add(new String[]{"GPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Mainboard
        if (computer.getMainboard() != null) {
            rows.add(new String[]{"Mainboard", "Name", computer.getMainboard().getName()});
            rows.add(new String[]{"", "Formfaktor", computer.getMainboard().getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getMainboard().getPrice())});
        } else {
            rows.add(new String[]{"Mainboard", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Ram
        if (computer.getRAM() != null) {
            rows.add(new String[]{"RAM", "Name", computer.getRAM().getName()});
            rows.add(new String[]{"", "Module", formatUtils.formatNumber(computer.getRamModule())});
            rows.add(new String[]{"", "Gesamtkapazität", formatUtils.formatNumber(getRAMCapacity(computer.getRamModule(), computer.getRAM().getCapacityGB())) + " GB"});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(computer.getRAM().getSpeedMHz()) + " MHz"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getRamModule() * computer.getRAM().getPrice())});

        } else {
            rows.add(new String[]{"RAM", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Netzteil
        if (computer.getPSU() != null) {
            rows.add(new String[]{"Netzteil", "Name", computer.getPSU().getName()});
            rows.add(new String[]{"", "Leistung", FormatUtils.formatNumber(computer.getPSU().getWattage()) + " W"});
            rows.add(new String[]{"", "Formfaktor", computer.getPSU().getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getPSU().getPrice())});
        } else {
            rows.add(new String[]{"Netzteil", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Storage
        List<Storage> storage = computer.getStorageDevices();
        if (storage != null && !storage.isEmpty()) {
            int i = 0;
            for (Storage s : storage) {
                String comp = (i == 0) ? "Speicher" : "";
                rows.add(new String[]{comp, "Name", s.getName()});
                rows.add(new String[]{"", "Kapazität", FormatUtils.formatNumber(s.getCapacityGB()) + " GB"});
                rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(s.getPrice())});
                rows.add(new String[]{"", "", ""});
                i++;
            }
        } else {
            rows.add(new String[]{"Speicher", "nicht ausgewählt", ""});
            rows.add(new String[]{"", "", ""});
        }

        // Trennlinie und Gesamtpreis
        rows.add(new String[]{"---", "---", "---"});
        rows.add(new String[]{"Leistungsaufnahme/Gesamt", "",
            computer.getPSU() != null
                ? FormatUtils.formatNumber(computer.getTotalPowerConsumption()) + " W / " + FormatUtils.formatNumber(computer.getPSU().getWattage()) + " W"
                : "nicht berechenbar"});
        rows.add(new String[]{"Gesamtpreis", "", FormatUtils.formatPrice(computer.getTotalPrice())});

        return rows.toArray(new String[0][]);
    }

    public static String[][] getDraftAsTableRows(ComputerDraft computer) {
    List<String[]> rows = new java.util.ArrayList<>();

        // Gehäuse
        if (computer.getComputerCase() != null) {
            rows.add(new String[]{"Gehäuse", "Name", computer.getComputerCase().getName()});
            rows.add(new String[]{"", "Formfaktor", computer.getComputerCase().getMotherboardFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(computer.getComputerCase().getPrice())});
        } else {
            rows.add(new String[]{"Gehäuse", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});
```



```java

        // CPU
        if (computer.getCPU() != null) {
            rows.add(new String[]{"CPU", "Name", computer.getCPU().getName()});
            rows.add(new String[]{"", "Kerne", FormatUtils.formatNumber(computer.getCPU().getCoreCount())});
            rows.add(new String[]{"", "Takt", formatUtils.formatNumber(computer.getCPU().getSpeedGHz()) + " GHz"});
            rows.add(new String[]{"", "Preis", formatUtils.formatPrice(computer.getCPU().getPrice())});
        } else {
            rows.add(new String[]{"CPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // GPU
        if (computer.getGPU() != null) {
            rows.add(new String[]{"GPU", "Name", computer.getGPU().getName()});
            rows.add(new String[]{"", "VRAM", formatUtils.formatNumber(computer.getGPU().getVramGB()) + " GB"});
            rows.add(new String[]{"", "Preis", formatUtils.formatPrice(computer.getGPU().getPrice())});
        } else {
            rows.add(new String[]{"GPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Mainboard
        if (computer.getMainboard() != null) {
            rows.add(new String[]{"Mainboard", "Name", computer.getMainboard().getName()});
            rows.add(new String[]{"", "Formfaktor", computer.getMainboard().getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", formatUtils.formatPrice(computer.getMainboard().getPrice())});
        } else {
            rows.add(new String[]{"Mainboard", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // RAM
        if (computer.getRAM() != null) {
            rows.add(new String[]{"RAM", "Name", computer.getRAM().getName()});
            rows.add(new String[]{"", "Module", formatUtils.formatNumber(computer.getRamModule())});
            rows.add(new String[]{"", "Gesamtkapazität", formatUtils.formatNumber(getRAMCapacity(computer.getRamModule(), computer.getRAM().getCapacityGB())) + " GB"});
            rows.add(new String[]{"", "Takt", formatUtils.formatNumber(computer.getRAM().getSpeedMHz()) + " MHz"});
            rows.add(new String[]{"", "Preis", formatUtils.formatPrice(computer.getRamModule() * computer.getRAM().getPrice())});
        } else {
            rows.add(new String[]{"RAM", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Netzteil
        if (computer.getPSU() != null) {
            rows.add(new String[]{"Netzteil", "Name", computer.getPSU().getName()});
            rows.add(new String[]{"", "Leistung", formatUtils.formatNumber(computer.getPSU().getWattage()) + " W"});
            rows.add(new String[]{"", "Formfaktor", computer.getPSU().getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", formatUtils.formatPrice(computer.getPSU().getPrice())});
        } else {
            rows.add(new String[]{"Netzteil", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Storage
        List<Storage> storage = computer.getStorage();
        if (storage != null && !storage.isEmpty()) {
            int i = 0;
            for (Storage s : storage) {
                String comp = (i == 0) ? "Speicher" : "";
                rows.add(new String[]{comp, "Name", s.getName()});
                rows.add(new String[]{"", "Kapazität", FormatUtils.formatNumber(s.getCapacityGB()) + " GB"});
                rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(s.getPrice())});
                rows.add(new String[]{"", "", ""});
                i++;
            }
        } else {
            rows.add(new String[]{"Speicher", "nicht ausgewählt", ""});
            rows.add(new String[]{"", "", ""});
        }

        // Trennlinie und Gesamtpreis
        rows.add(new String[]{"---", "---", "---"});
        rows.add(new String[]{"Leistungsaufnahme/Gesamt", "",
        FormatUtils.formatNumber(computer.getTotalPowerConsumption()) + " W / " +
          FormatUtils.formatNumber(computer.getPSU() == null ? 0 : computer.getPSU().getWattage()) + " W"});
        rows.add(new String[]{"Gesamtpreis", "", FormatUtils.formatPrice(computer.getTotalPrice())});

        return rows.toArray(new String[0][]);
    }

    private static int getRAMCapacity(int module, int capacity) {
        return module* capacity;
    }

}
```

4.2 DRY - Nachher

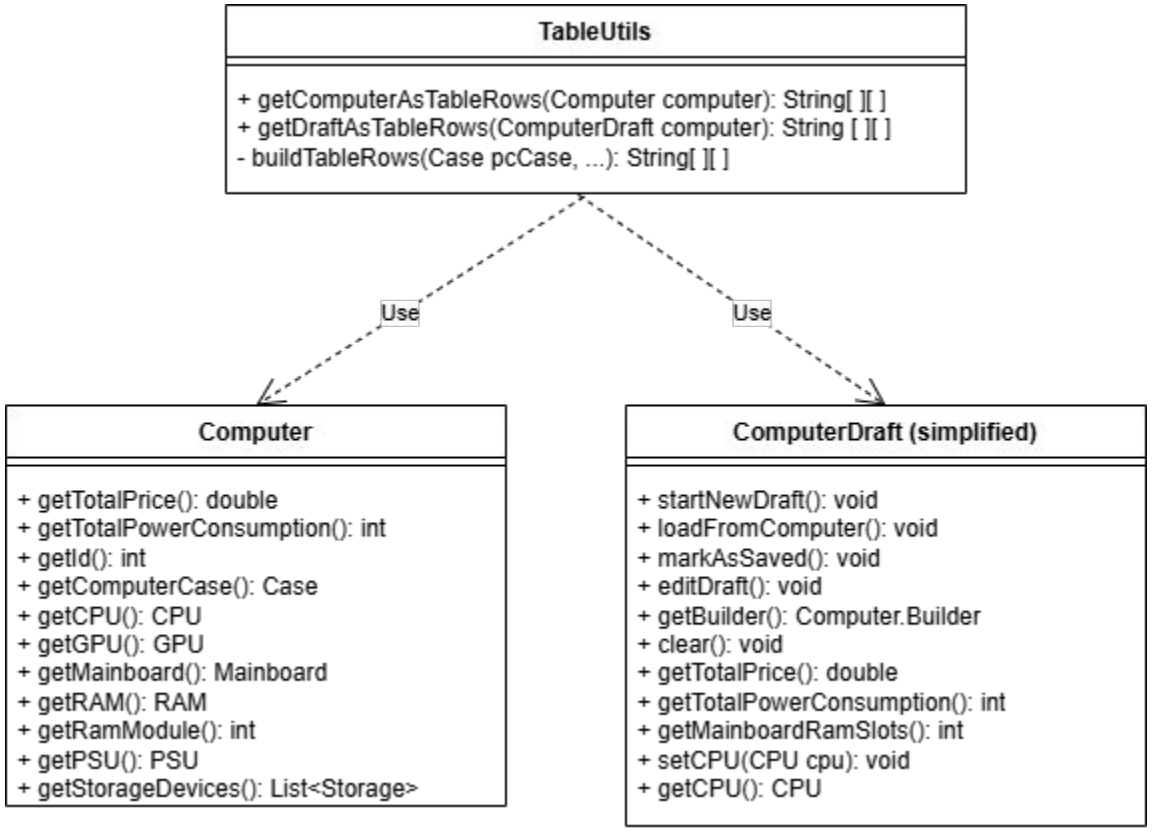
TableUtils.java

Commit: 72507d2

```java
package de.ase.pcpartpicker.adapters.cli.utils;

import java.util.List;

import de.ase.pcpartpicker.adapters.cli.ComputerDraft;
import de.ase.pcpartpicker.domain.CPU;
import de.ase.pcpartpicker.domain.Case;
import de.ase.pcpartpicker.domain.GPU;
import de.ase.pcpartpicker.domain.Mainboard;
import de.ase.pcpartpicker.domain.PSU;
import de.ase.pcpartpicker.domain.RAM;
import de.ase.pcpartpicker.domain.Storage;
import de.ase.pcpartpicker.part_assembly.Computer;

public class TableUtils {

    public static String[][] getComputerAsTableRows(Computer computer) {
        return buildTableRows(
            computer.getComputerCase(), computer.getCPU(), computer.getGPU(),
            computer.getMainboard(), computer.getRAM(), computer.getRamModule(),
            computer.getPSU(), computer.getStorageDevices(),
            computer.getTotalPowerConsumption(), computer.getTotalPrice()
        );
    }

    // 2. Methode für den unfertigen ComputerDraft
    public static String[][] getDraftAsTableRows(ComputerDraft draft) {
        return buildTableRows(
            draft.getComputerCase(), draft.getCPU(), draft.getGPU(),
            draft.getMainboard(), draft.getRAM(), draft.getRamModule(),
            draft.getPSU(), draft.getStorage(),
            draft.getTotalPowerConsumption(), draft.getTotalPrice()
        );
    }

    private static String[][] buildTableRows(Case pcCase, CPU cpu, GPU gpu, Mainboard mb, RAM ram, int ramModule, PSU psu, List<Storage> storage, int totalPower, double totalPrice) {
        List<String[]> rows = new java.util.ArrayList<>();

        // Gehäuse
        if (pcCase != null) {
            rows.add(new String[]{"Gehäuse", "Name", pcCase.getName()});
            rows.add(new String[]{"", "Formfaktor", pcCase.getMotherboardFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(pcCase.getPrice())});
        } else {
            rows.add(new String[]{"Gehäuse", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // CPU
        if (cpu != null) {
            rows.add(new String[]{"CPU", "Name", cpu.getName()});
            rows.add(new String[]{"", "Kerne", FormatUtils.formatNumber(cpu.getCoreCount())});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(cpu.getSpeedGHz()) + " GHz"}); // Tippfehler bei dir korrigiert (formatNumer -> formatNumber)
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(cpu.getPrice())});
        } else {
            rows.add(new String[]{"CPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // GPU
        if (gpu != null) {
            rows.add(new String[]{"GPU", "Name", gpu.getName()});
            rows.add(new String[]{"", "VRAM", FormatUtils.formatNumber(gpu.getVramGB()) + " GB"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(gpu.getPrice())});
        } else {
            rows.add(new String[]{"GPU", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Mainboard
        if (mb != null) {
            rows.add(new String[]{"Mainboard", "Name", mb.getName()});
            rows.add(new String[]{"", "Formfaktor", mb.getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(mb.getPrice())});
        } else {
            rows.add(new String[]{"Mainboard", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});


```

```java

        // RAM
        if (ram != null) {
            rows.add(new String[]{"RAM", "Name", ram.getName()});
            rows.add(new String[]{"", "Module", FormatUtils.formatNumber(ramModule)});
            rows.add(new String[]{"", "Gesamtkapazität", FormatUtils.formatNumber(getRAMCapacity(ramModule, ram.getCapacityGB())) + " GB"});
            rows.add(new String[]{"", "Takt", FormatUtils.formatNumber(ram.getSpeedMHz()) + " MHz"});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(ramModule * ram.getPrice())});
        } else {
            rows.add(new String[]{"RAM", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Netzteil
        if (psu != null) {
            rows.add(new String[]{"Netzteil", "Name", psu.getName()});
            rows.add(new String[]{"", "Leistung", FormatUtils.formatNumber(psu.getWattage()) + " W"});
            rows.add(new String[]{"", "Formfaktor", psu.getFormFactor().getName()});
            rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(psu.getPrice())});
        } else {
            rows.add(new String[]{"Netzteil", "nicht ausgewählt", ""});
        }
        rows.add(new String[]{"", "", ""});

        // Storage
        if (storage != null && !storage.isEmpty()) {
            int i = 0;
            for (Storage s : storage) {
                String comp = (i == 0) ? "Speicher" : "";
                rows.add(new String[]{comp, "Name", s.getName()});
                rows.add(new String[]{"", "Kapazität", FormatUtils.formatNumber(s.getCapacityGB()) + " GB"});
                rows.add(new String[]{"", "Preis", FormatUtils.formatPrice(s.getPrice())});
                rows.add(new String[]{"", "", ""});
                i++;
            }
        } else {
            rows.add(new String[]{"Speicher", "nicht ausgewählt", ""});
            rows.add(new String[]{"", "", ""});
        }

        // Trennlinie und Gesamtpreis (Logik vereint)
        rows.add(new String[]{"---", "---", "---"});
        String psuWattage = psu != null ? FormatUtils.formatNumber(psu.getWattage()) + " W" : "0 W";
        rows.add(new String[]{"Leistungsaufnahme/Gesamt", "", FormatUtils.formatNumber(totalPower) + " W / " + psuWattage});
        rows.add(new String[]{"Gesamtpreis", "", FormatUtils.formatPrice(totalPrice)});

        return rows.toArray(new String[0][]);
    }

    private static int getRAMCapacity(int module, int capacity) {
        return module * capacity;
    }
}

```

4.2 DRY – UML


TableUtils
+ getComputerAsTableRows(Computer computer): String[ ][ ]
+ getDraftAsTableRows(ComputerDraft computer): String [ ][ ]
- buildTableRows(Case pcCase, ...): String[ ][ ]
Use
Use
Computer
+ getTotalPrice(): double
+ getTotalPowerConsumption(): int
+ getId(): int
+ getComputerCase(): Case
+ getCPU(): CPU
+ getGPU(): GPU
+ getMainboard(): Mainboard
+ getRAM(): RAM
+ getRamModule(): int
+ getPSU(): PSU
+ getStorageDevices(): List<Storage>
ComputerDraft (simplified)
+ startNewDraft(): void
+ loadFromComputer(): void
+ markAsSaved(): void
+ editDraft(): void
+ getBuilder(): Computer.Builder
+ clear(): void
+ getTotalPrice(): double
+ getTotalPowerConsumption(): int
+ getMainboardRamSlots(): int
+ setCPU(CPU cpu): void
+ getCPU(): CPU

5. Unit Tests – Menu Tests

```java
@Test
    @DisplayName("Testet, ob ShowListCommand eine Liste ausgibt")
    void showListCommandPrintsList() {
        ByteArrayOutputStream outContent = new ByteArrayOutputStream();
        PrintStream originalOut = System.out;
        System.setOut(new PrintStream(outContent));

        String simulatedInput = "0\n";
        System.setIn(new java.io.ByteArrayInputStream(simulatedInput.getBytes()));

        ComponentRepository<Component> dummyRepo = new ComponentRepository<Component>() {
            @Override
            public java.util.List<Component> findAll() {
                return java.util.Collections.emptyList();
            }
        };

        rListConfiguration<Component> config = new rListConfiguration<>(
            "Test-Komponenten",
            new String[] { "Header1", "Header2" },
            dummyRepo,
            c -> new String[] { "dummy" }
        );

        InputReader reader = new InputReader();
        ShowListCommand<Component> command = new ShowListCommand<>(config, reader);
        command.render("Simulation");

        System.setOut(originalOut);
        String output = outContent.toString();
        assertTrue(output.contains("Test-Komponenten"), "Die Komponententitel sollten ausgegeben werden.");
    }
```

5. Unit Tests – Menu Tests

```java
@Test
    @DisplayName("Testet, ob ExitCommand das Runnable ausführt und 'Auf Wiedersehen!' ausgibt")
    void exitCommandExecutes() {
        final boolean[] wasRun = { false };
        Runnable onExit = () -> wasRun[0] = true;

        ByteArrayOutputStream outContent = new ByteArrayOutputStream();
        PrintStream originalOut = System.out;
        System.setOut(new PrintStream(outContent));

        ExitCommand exitCommand = new ExitCommand(onExit);
        exitCommand.execute();

        System.setOut(originalOut);
        String output = outContent.toString();

        assertTrue(wasRun[0], "Das Runnable sollte ausgeführt worden sein.");
        assertTrue(output.contains("Auf Wiedersehen!"), "Die Abschiedsnachricht sollte ausgegeben werden.");
    }
```

5. Unit Tests – Menu Tests

```java
@Test
    @DisplayName("Testet, ob BackCommand ausführbar ist")
    void backCommandExecutes() {
        final boolean[] wasRun = { false };
        Runnable onBack = () -> wasRun[0] = true;

        BackCommand backCommand = new BackCommand(onBack);
        backCommand.execute();

        assertTrue(wasRun[0], "Das Runnable sollte ausgeführt worden sein.");

    }
```

5. Unit Tests – Menu Tests

```java
@Test
    @DisplayName("Testet, ob ein Menü geöffnet werden kann")
    void openMenuCommandOpensMenu() {
        DummyMenu menu = new DummyMenu("TestMenu", new InputReader());
        OpenMenuCommand command = new OpenMenuCommand(menu);
        command.execute();
        assertTrue(menu.opened, "Das Menü sollte geöffnet worden sein.");
    }
```

5. Unit Tests – Database Tests

```java
@Test
    void userRepositoryCanSaveAndLoadUser() {
        UserRepository userRepository = new UserRepository(connectionFactory);
        String userName = "Integration Test User " + System.nanoTime();

        User savedUser = userRepository.save(userName);
        assertTrue(savedUser.getId() > 0);

        User loadedUser = userRepository.findById(savedUser.getId());
        assertNotNull(loadedUser);
        assertEquals(savedUser.getId(), loadedUser.getId());
        assertEquals(userName, loadedUser.getName());
    }
```

5. Unit Tests – Database Tests

```java
@Test
    void computerRepositoryCanSaveAndLoadByUserId() {
        UserRepository userRepository = new UserRepository(connectionFactory);
        ComputerRepository computerRepository = new ComputerRepository(connectionFactory);

        User user = userRepository.save("Computer Repo Test User " + System.nanoTime());
        Computer computer = createCompatibleComputer(connectionFactory);

        int savedComputerId = computerRepository.save(user.getId(), computer);
        assertTrue(savedComputerId > 0);

        List<Computer> userComputers = computerRepository.findAllByUserId(user.getId());
        assertEquals(1, userComputers.size());

        Computer loadedComputer = userComputers.get(0);
        assertEquals(computer.getCPU().getId(), loadedComputer.getCPU().getId());
        assertEquals(computer.getMainboard().getId(), loadedComputer.getMainboard().getId());
        assertEquals(computer.getRAM().getId(), loadedComputer.getRAM().getId());
        assertEquals(computer.getPSU().getId(), loadedComputer.getPSU().getId());
        assertEquals(computer.getComputerCase().getId(), loadedComputer.getComputerCase().getId());
    }
```

5. Unit Tests – Domain Tests

```java
@Test
    public void testGPUCreation() {
        GPU gpu = new GPU(1, "Test GPU", 299.99, new Manufacturer(1, "Test Manufacturer"), 2220, 3290.0, 16, 190);
        assertNotNull(gpu);
        assertEquals("Test GPU", gpu.getName());
        assertEquals(299.99, gpu.getPrice());
        assertEquals("Test Manufacturer", gpu.getManufacturer().getName());
        assertEquals(2220, gpu.getCoreClockMHz());
        assertEquals(3290.0, gpu.getBoostClockMHz());
        assertEquals(16, gpu.getVramGB());
        assertEquals(190, gpu.getPowerConsumptionW());
    }
```

5. Unit Tests – Computer Tests

```java
@Test
    public void testComputerCreation() {

        PSUFormFactor psuFormFactor = new PSUFormFactor(0, "SFTX");
        MotherboardFormFactor motherboardFormFactor = new MotherboardFormFactor(34, "ATX");
        Socket socket = new Socket(1, "AM4");

        Computer computer = new Computer.Builder()
            .setCPU(new CPU(1, "Test CPU", 199.99, new Manufacturer(1, "Test Manufacturer"), socket, 8, false, 95))
            .setGPU(new GPU(1, "Test GPU", 299.99, new Manufacturer(1, "Test Manufacturer"), 16, 150))
            .setMainboard(new Mainboard(1, "Test Mainboard", 149.99, new Manufacturer(1, "Test Manufacturer"), socket, motherboardFormFactor, 4, 3, 6, 2))
            .setRAM(new RAM(1, "Test RAM", 79.99, new Manufacturer(1, "Test Manufacturer"), 16, 15), 2)
            .setPSU(new PSU(1, "Test PSU", 89.99, new Manufacturer(1, "Test Manufacturer"), 650, psuFormFactor))
            .setComputerCase(new Case(1, "Test Case", 59.99, new Manufacturer(1, "Test Manufacturer"), motherboardFormFactor, psuFormFactor, true, 9))
            .setStorageDevices(new Storage[] {
                new SSD(1, "Test SSD", 129.99, new Manufacturer(1, "Test Manufacturer"), 512),
                new HDD(2, "Test HDD", 79.99, new Manufacturer(1, "Test Manufacturer"), 1024)
            })
            .build();

        assertNotNull(computer);
        //computer.printConfiguration();
    }
```

5. Unit Tests – Computer Tests

```java
@Test
    public void testWrongComputer() {
        Computer computer = new Computer.Builder()
            .setCPU(null)
            .setGPU(null)
            .setMainboard(null)
            .setRAM(null, 0)
            .setPSU(null)
            .setComputerCase(null)
            .setStorageDevices(new Storage[] {})
            .build();

        assertNull(computer);
    }
```

5. Unit Tests – User Tests

```java
@Test
    public void loginTest() {
        String username = "login-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login -> Starten -> Username -> Enter -> Zurück
            "0"                          // Beenden Hauptmenü
        ) + "\n";

        String output = runTestFlow(flowInput);

        assertTrue(SessionManager.isLoggedIn());
        assertEquals(username, SessionManager.getcurrentUser().getName());
        assertTrue(output.contains("Willkommen zurück"));
    }

```

5.1 ATRIP - Automatic

- Automatisierung Benutzereingaben
  - ByteArrayInputStream

```java
@Test
    public void loginTest() {
        String username = "login-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login -> Starten -> Username -> Enter -> Zurück
            "0"                          // Beenden Hauptmenü
        ) + "\n";

        String output = runTestFlow(flowInput);

        assertTrue(SessionManager.isLoggedIn());
        assertEquals(username, SessionManager.getcurrentUser().getName());
        assertTrue(output.contains("Willkommen zurück"));
    }

```

5.1 ATRIP - Automatic

- Automatisches Setup mit @BeforeEach z.B. bei UserTests

```java
@BeforeEach
    public void setupEach() {
        originalIn = System.in;
        originalOut = System.out;

        userRepository = new InMemoryUserRepository();
        computerRepository = new InMemoryComputerRepository();

        SessionManager.setCurrentUser(null);
        createCompatibleTestComponents();
    }
```

5.1 ATRIP - Automatic

- Automatisches „Tear-Down“, sodass der Originalzustand wiederhergestellt wird

```
@AfterEach
    public void tearDownEach() {
        SessionManager.setCurrentUser(null);
        System.setIn(originalIn);
        System.setOut(originalOut);
    }
```

5.1 ATRIP - Automatic

- Selbständige Bewertung der Tests durch Assertions

```java
@Test
    public void loginTest() {
        String username = "login-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login -> Starten -> Username -> Enter -> Zurück
            "0"                          // Beenden Hauptmenü
        ) + "\n";

        String output = runTestFlow(flowInput);

        assertTrue(SessionManager.isLoggedIn());
        assertEquals(username, SessionManager.getcurrentUser().getName());
        assertTrue(output.contains("Willkommen zurück"));
    }

```

5.1 ATRIP – Thorough

- Testen von korrekten, als auch inkorrekten Eingaben

```java
@Test
    public void createCorrectComputer() {
        String username = "builder-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login
            "2", "1",                    // Computerverwaltung -> Neu anlegen
            "1", "1", "", "0",           // CPU -> ID 1 -> Enter -> Zurück Paging
            "2", "1", "", "0",           // GPU -> ID 1 -> Enter -> Zurück Paging
            "3", "1", "", "0",           // Mainboard -> ID 1 -> Enter -> Zurück Paging
            "4", "1", "4", "", "0",      // RAM -> ID 1 -> 4 Module -> Enter -> Zurück Paging
            "5", "1", "", "0",           // PSU -> ID 1 -> Enter -> Zurück Paging
            "6", "1", "", "0",           // Case -> ID 1 -> Enter -> Zurück Paging
            "7", "1", "", "0",           // SSD -> ID 1 -> Enter -> Zurück Paging
            "8", "",                     // Finish + waitForEnter
            "0",                         // Zurueck aus Konfigurator
            "0",                         // Zurueck aus Computerverwaltung
            "0"                          // Beenden
        ) + "\n";

        String output = runTestFlow(flowInput);

        User currentUser = SessionManager.getcurrentUser();
        assertEquals(1, computerRepository.findAllByUserId(currentUser.getId()).size());
        assertTrue(output.contains("ERFOLG"));
    }
```

```java
@Test
    public void createWrongComputer() {
        String username = "wrong-build-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login
            "2", "1",                    // Computerverwaltung -> Neu anlegen
            "8", "",                     // Versuch speichern ohne Komponenten -> Print Error & Enter
            "0",                         // Zurueck Konfigurator
            "0",                         // Zurueck Computerverwaltung
            "0"                          // Beenden
        ) + "\n";

        String output = runTestFlow(flowInput);

        User currentUser = SessionManager.getcurrentUser();
        assertTrue(computerRepository.findAllByUserId(currentUser.getId()).isEmpty());
        assertTrue(output.contains("FEHLER"));
    }
```

5.1 ATRIP – Thorough

pc-konfigurator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
de.ase.pcpartpicker.adapters.cli.commands		40 %		31 %	128	191	335	553	44	83	5	23
de.ase.pcpartpicker.adapters.cli		63 %		43 %	116	212	180	513	49	128	1	12
de.ase.pcpartpicker.part_assembly		74 %		55 %	93	165	86	324	4	40	1	6
de.ase.pcpartpicker.adapters.sqlite		84 %		60 %	86	181	103	506	3	50	0	4
de.ase.pcpartpicker.adapters.cli.utils		80 %		43 %	65	107	59	220	11	38	0	7
de.ase.pcpartpicker.adapters.sqlite.repositories		85 %		69 %	45	143	50	391	12	83	0	14
de.ase.pcpartpicker.domain		89 %		47 %	23	72	6	150	3	52	0	11
de.ase.pcpartpicker		0 %		0 %	9	9	17	17	8	8	2	2
de.ase.pcpartpicker.domain.HelperClasses		100 %		90 %	1	15	0	23	0	10	0	6
Total	3.919 of 13.716	71 %	590 of 1.194	50 %	566	1.095	836	2.697	134	492	9	85

5.1 ATRIP - Professional

- Test folgen denselben Prinzipien wie der restliche Code
  - Arrange, Act, Assert (Vorbereiten, Ausführen, Überprüfen)

```java
@BeforeEach
    public void setupEach() {
        originalIn = System.in;
        originalOut = System.out;

        userRepository = new InMemoryUserRepository();
        computerRepository = new InMemoryComputerRepository();

        SessionManager.setCurrentUser(null);
        createCompatibleTestComponents();
    }
```

```java
@Test
    public void loginTest() {
        String username = "login-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login -> Starten -> Username -> Enter -> Zurück
            "0"                          // Beenden Hauptmenü
        ) + "\n";

        String output = runTestFlow(flowInput);

        assertTrue(SessionManager.isLoggedIn());
        assertEquals(username, SessionManager.getcurrentUser().getName());
        assertTrue(output.contains("Willkommen zurück"));
    }

```

```java
@AfterEach
    public void tearDownEach() {
        SessionManager.setCurrentUser(null);
        System.setIn(originalIn);
        System.setOut(originalOut);
    }
```

5.1 ATRIP - Professional

- Test folgen dem DRY Prinzip

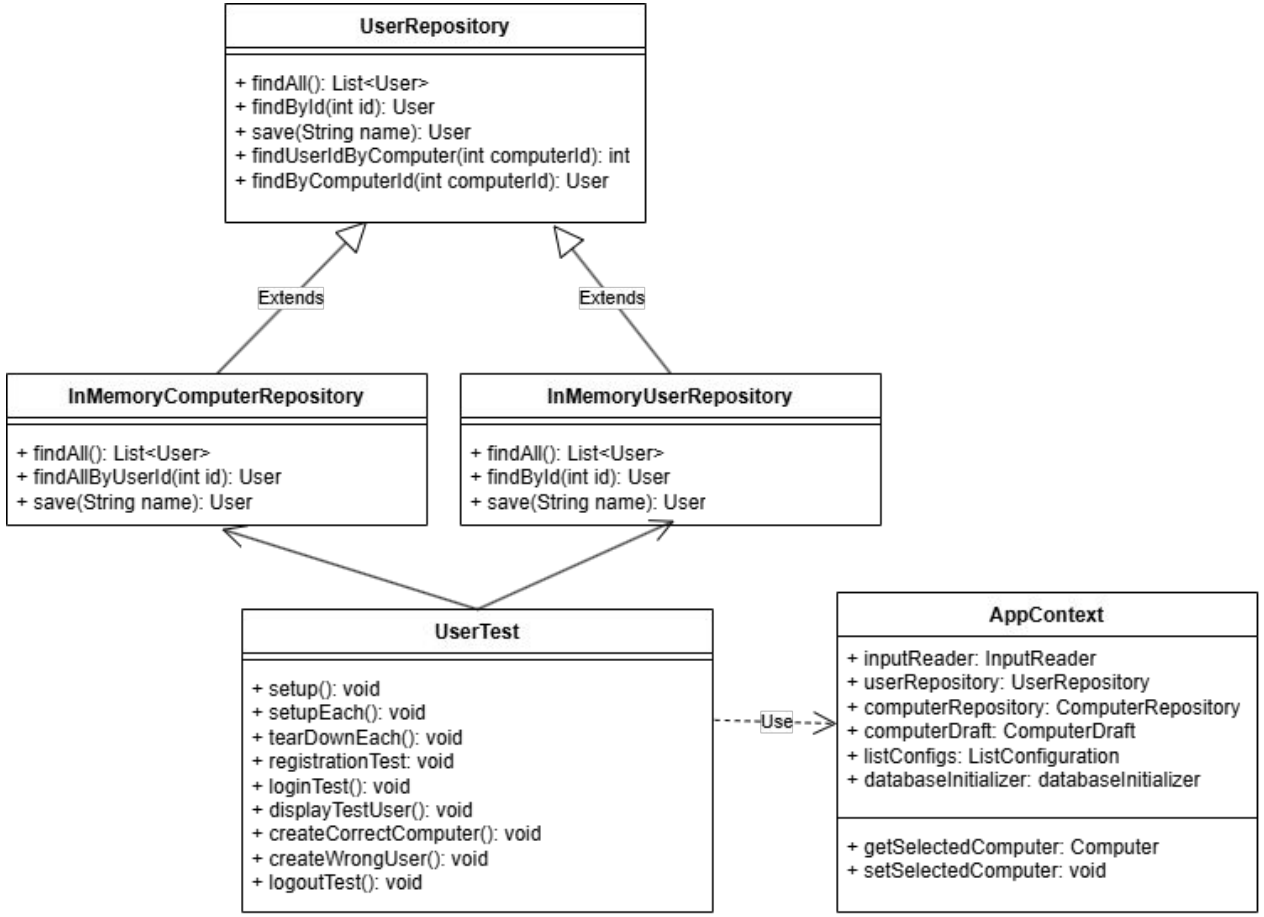
```java
private String runTestFlow(String flowInput) {
        ByteArrayOutputStream outContent = new ByteArrayOutputStream();
        System.setOut(new PrintStream(outContent));
        System.setIn(new ByteArrayInputStream(flowInput.getBytes(StandardCharsets.UTF_8)));

        context = new AppContext();

        try {
            java.lang.reflect.Field userRepoField = AppContext.class.getDeclaredField("userRepository");
            userRepoField.setAccessible(true);
            userRepoField.set(context, userRepository);

            java.lang.reflect.Field compRepoField = AppContext.class.getDeclaredField("computerRepository");
            compRepoField.setAccessible(true);
            compRepoField.set(context, computerRepository);

            // WICHTIG: Sicherstellen, dass computerDraft im Context existiert
            java.lang.reflect.Field draftField = AppContext.class.getDeclaredField("computerDraft");
            draftField.setAccessible(true);
            if (draftField.get(context) == null) {
                draftField.set(context, new ComputerDraft());
            }

            // Sicherstellen, dass inputReader da ist
            java.lang.reflect.Field readerField = AppContext.class.getDeclaredField("inputReader");
            readerField.setAccessible(true);
            if (readerField.get(context) == null) {
                readerField.set(context, new InputReader());
            }
        } catch (Exception e) {
            e.printStackTrace();
        }

        InputReader currentReader = null;
        try {
            java.lang.reflect.Field readerField = AppContext.class.getDeclaredField("inputReader");
            readerField.setAccessible(true);
            currentReader = (InputReader) readerField.get(context);
        } catch(Exception e) {}

        createMainMenu(currentReader).execute();

        return outContent.toString(StandardCharsets.UTF_8);
    }
```

5.1 ATRIP - Professional

- Lesbarkeit durch Kommentare erhöhen

```java
@Test
    public void loginTest() {
        String username = "login-user";

        String flowInput = String.join("\n",
            "3", "1", username, "", "0", // Nutzer anlegen
            "4", "1", username, "", "0", // Login -> Starten -> Username -> Enter -> Zurück
            "0"                          // Beenden Hauptmenü
        ) + "\n";

        String output = runTestFlow(flowInput);

        assertTrue(SessionManager.isLoggedIn());
        assertEquals(username, SessionManager.getcurrentUser().getName());
        assertTrue(output.contains("Willkommen zurück"));
    }

```

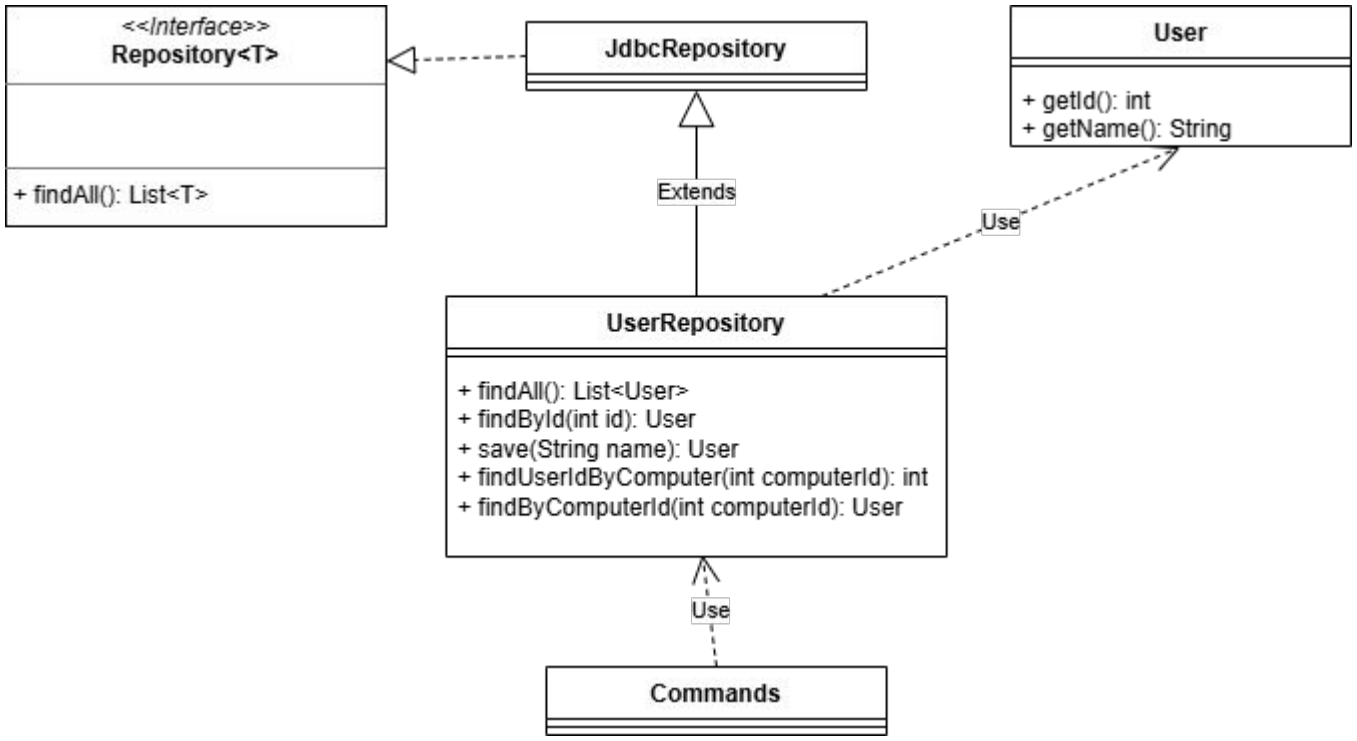
5.2 Fakes und Mocks - UML



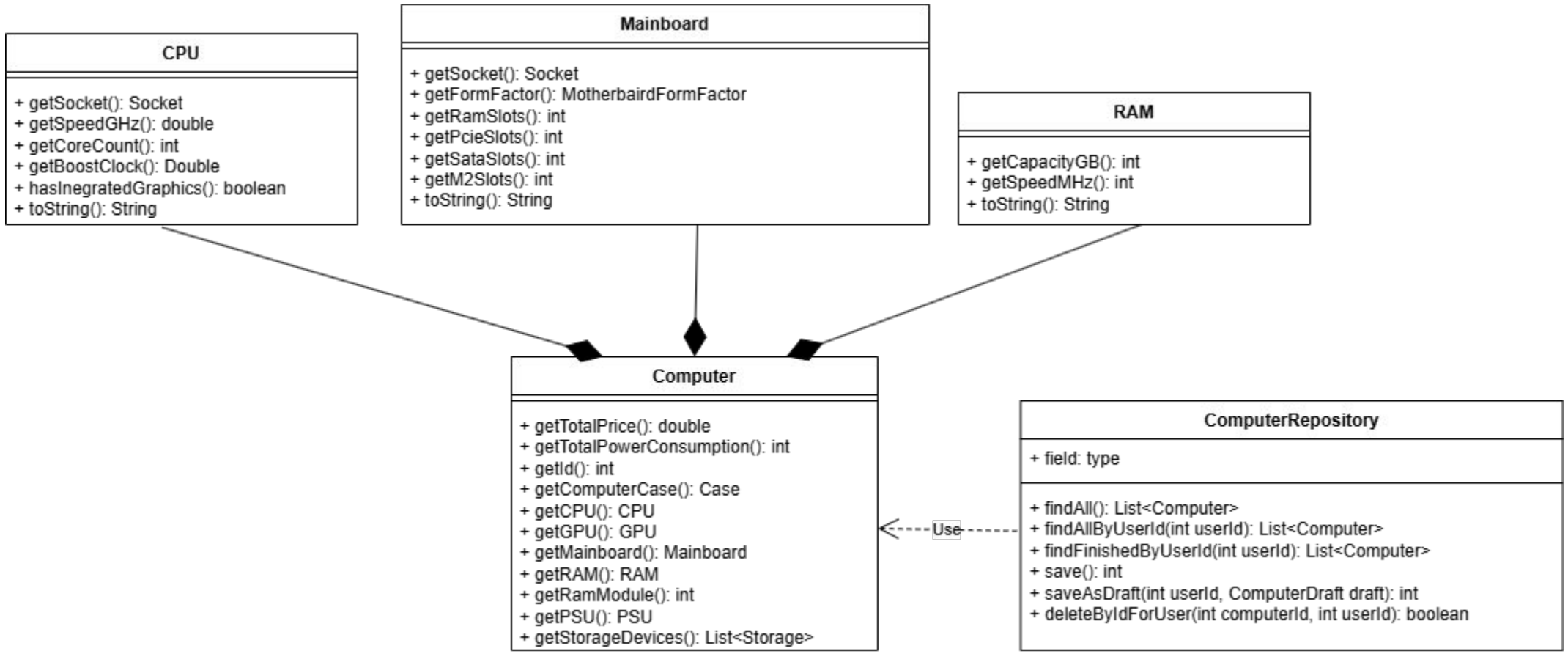
6. Domain Driven Design (DDD) - Ubiquitous Language

- ComputerDraft
- MotherboardFormFactor
- PowerConsumption
  - getWattage
- Bottleneck-Check

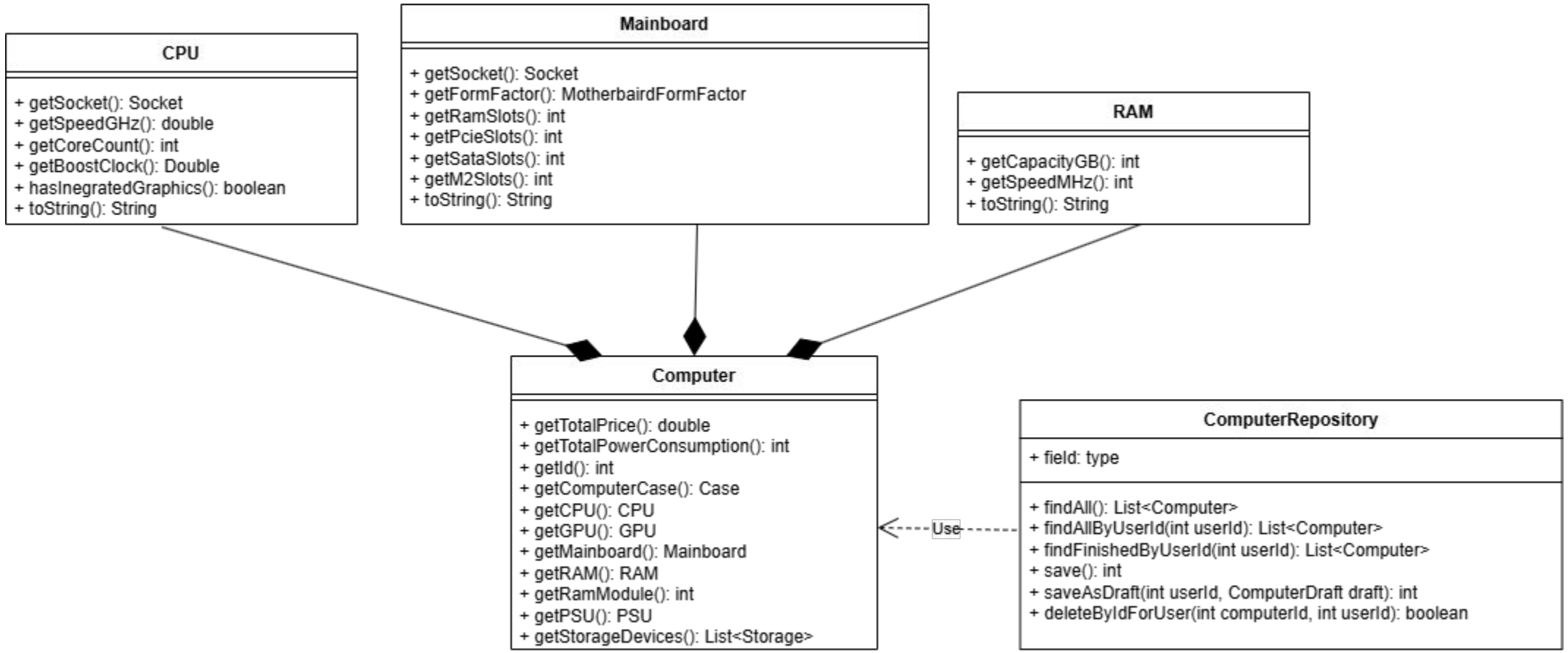
6.1 DDD – Repositories – UML



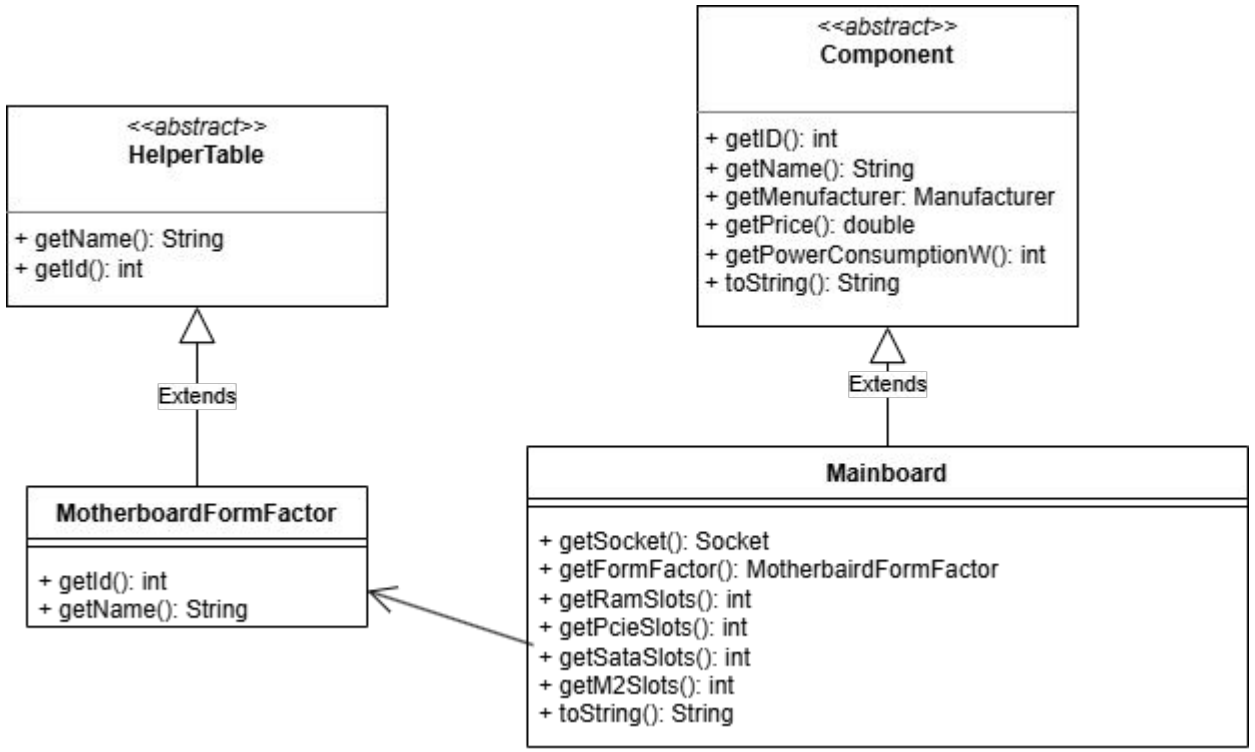
6.2 DDD – Aggregates



6.3 DDD – Entities



6.4 DDD – Value Objects



7. Refactoring – Code Smells – Long Method

```java
/**
     * Diese Methode überprüft die Kompatibilität der Komponenten.
     * Es wird lediglich überprüft, ob der PC so gebaut werden und existieren könnte.
     * Bspw. ob das Netzteil genug Leistung hat, um die Komponenten zu versorgen, wird hier nicht überprüft.
     * @return true, wenn die Komponenten kompatibel sind und der Computer erstellt werden kann, andernfalls false. Es werden auch Fehlermeldungen ausgegeben, die erklären, warum der Computer nicht erstellt werden kann.
     * @author Tuluhan
     */
    private boolean validate() {

        if(ram == null) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss mindestens ein RAM-Modul ausgewählt werden.");
            return false;
        }

        if(cpu == null) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss eine CPU ausgewählt werden.");
            return false;
        }

        if(!cpu.hasIntegratedGraphics() && gpu == null) {
            System.out.println(ColorConstants.YELLOW("WARNUNG") + " | Der Computer hat keine dedizierte Grafikkarte und die CPU verfügt nicht über integrierte Grafik. Es könnte zu Problemen bei der Anzeige kommen.");
        }

        if(mainboard == null) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Mainboard ausgewählt werden.");
            return false;
        }

        if(psu == null) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Netzteil ausgewählt werden.");
            return false;
        }

        if(computerCase == null) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss ein Gehäuse ausgewählt werden.");
            return false;
        }

        if (!computerCase.getPSUFormFactor().equals(psu.getFormFactor())) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Die Form des Netzteils passt nicht zum Gehäuse.");
            return false;
        }

        if (!checkMainboardCaseCompatibility(mainboard, computerCase)) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Das Mainboard passt nicht ins Gehäuse.");
            return false;
        }

        if(!cpu.getSocket().equals(mainboard.getSocket())) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Die CPU ist nicht mit dem Mainboard kompatibel.");
            return false;
        }

        if(ramModule > mainboard.getRamSlots()) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Das Mainboard hat nicht genug RAM-Slots für die Anzahl der RAM-Module.");
            return false;
        }

        if(storageDevices.isEmpty()) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Es muss mindestens ein Speichermedium ausgewählt werden.");
            return false;
        }

        if(psu.getWattage() < calculateMomentaryPowerConsumption()) {
            System.out.println(ColorConstants.RED("FEHLER") + " | Das Netzteil hat nicht genug Leistung um das System zu versorgen. Das Netzteil muss mindestens " + calculateMomentaryPowerConsumption() + "W liefern, um alle Komponenten zu versorgen.");
            return false;
        }

        return true;
    }
```

7. Refactoring – Code Smells – Long Method - Lösung

```java
private boolean validate() {
    if (!hasAllRequiredComponents()) return false;
    if (!areFormFactorsCompatible()) return false;
    if (!isHardwareCompatible())     return false;
    if (!hasSufficientPower())       return false;

    printOptionalWarnings(); // z.B. Hinweis auf fehlende GPU
    return true;
}

// --- Die extrahierten Hilfsmethoden ---

private boolean hasAllRequiredComponents() {
    if (ram == null || cpu == null || mainboard == null || psu == null || computerCase == null || storageDevices.isEmpty()) {
        System.out.println("FEHLER | Es fehlen zwingend erforderliche Komponenten.");
        return false;
    }
    return true;
}

private boolean areFormFactorsCompatible() {
    if (!computerCase.getPSUFormFactor().equals(psu.getFormFactor())) {
        System.out.println("FEHLER | Die Form des Netzteils passt nicht zum Gehäuse.");
        return false;
    }
    if (!checkMainboardCaseCompatibility(mainboard, computerCase)) {
        System.out.println("FEHLER | Das Mainboard passt nicht ins Gehäuse.");
        return false;
    }
    return true;
}

private boolean isHardwareCompatible() {
    if (!cpu.getSocket().equals(mainboard.getSocket())) {
        System.out.println("FEHLER | Die CPU ist nicht mit dem Mainboard kompatibel.");
        return false;
    }
    if (ramModule > mainboard.getRamSlots()) {
        System.out.println("FEHLER | Zu viele RAM-Module für dieses Mainboard.");
        return false;
    }
    return true;
}

private boolean hasSufficientPower() {
    if (psu.getWattage() < calculateMomentaryPowerConsumption()) {
        System.out.println("FEHLER | Das Netzteil hat nicht genug Leistung.");
        return false;
    }
    return true;
}
```

7. Refactoring – Code Smells – Long Parameter List

TableUtils.java

```
1       private static String[][] buildTableRows(Case pcCase, CPU cpu, GPU gpu, Mainboard mb, RAM ram, int ramModule, PSU psu, List<Storage> storage, int totalPower, double totalPrice) {
2
```

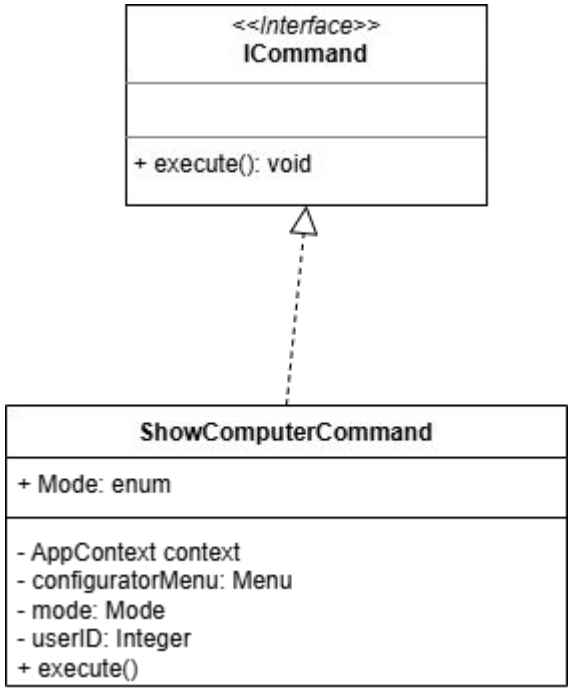
7. Refactoring – Code Smells – Long Parameter List – Lösung

```java
public interface IComputerData {
    Case getComputerCase();
    CPU getCPU();
    GPU getGPU();
    Mainboard getMainboard();
    RAM getRAM();
    int getRamModule();
    PSU getPSU();
    List<Storage> getStorage();
    int getTotalPowerConsumption();
    double getTotalPrice();
}

public class Computer implements IComputerData { /* ... */ }
public class ComputerDraft implements IComputerData { /* ... */ }

public class TableUtils {

    public static String[][] getComputerAsTableRows(Computer computer) {
        return buildTableRows(computer);
    }

    public static String[][] getDraftAsTableRows(ComputerDraft draft) {
        return buildTableRows(draft);
    }

    private static String[][] buildTableRows(IComputerData data) {
        List<String[]> rows = new ArrayList<>();

        if (data.getCPU() != null) {
            rows.add(new String[]{"CPU", "Name", data.getCPU().getName()});
        }

        return rows.toArray(new String[0][]);
    }
}
```

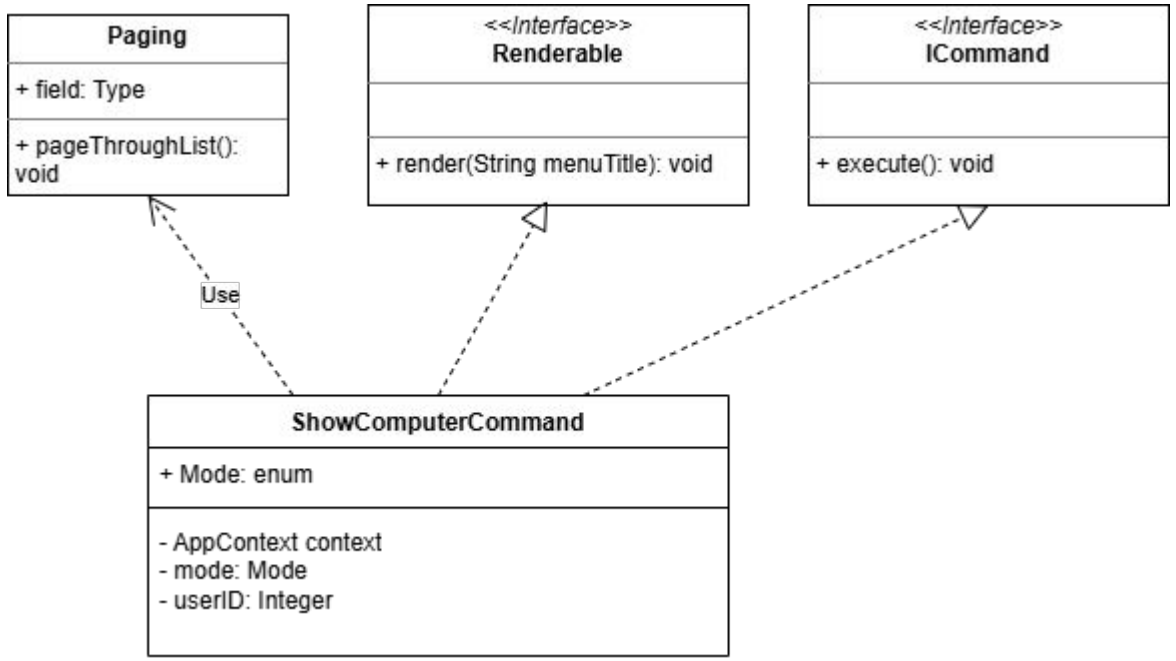
7.1 Refactoring – Extract Class – UML

Commit: fed4d10

Vorher

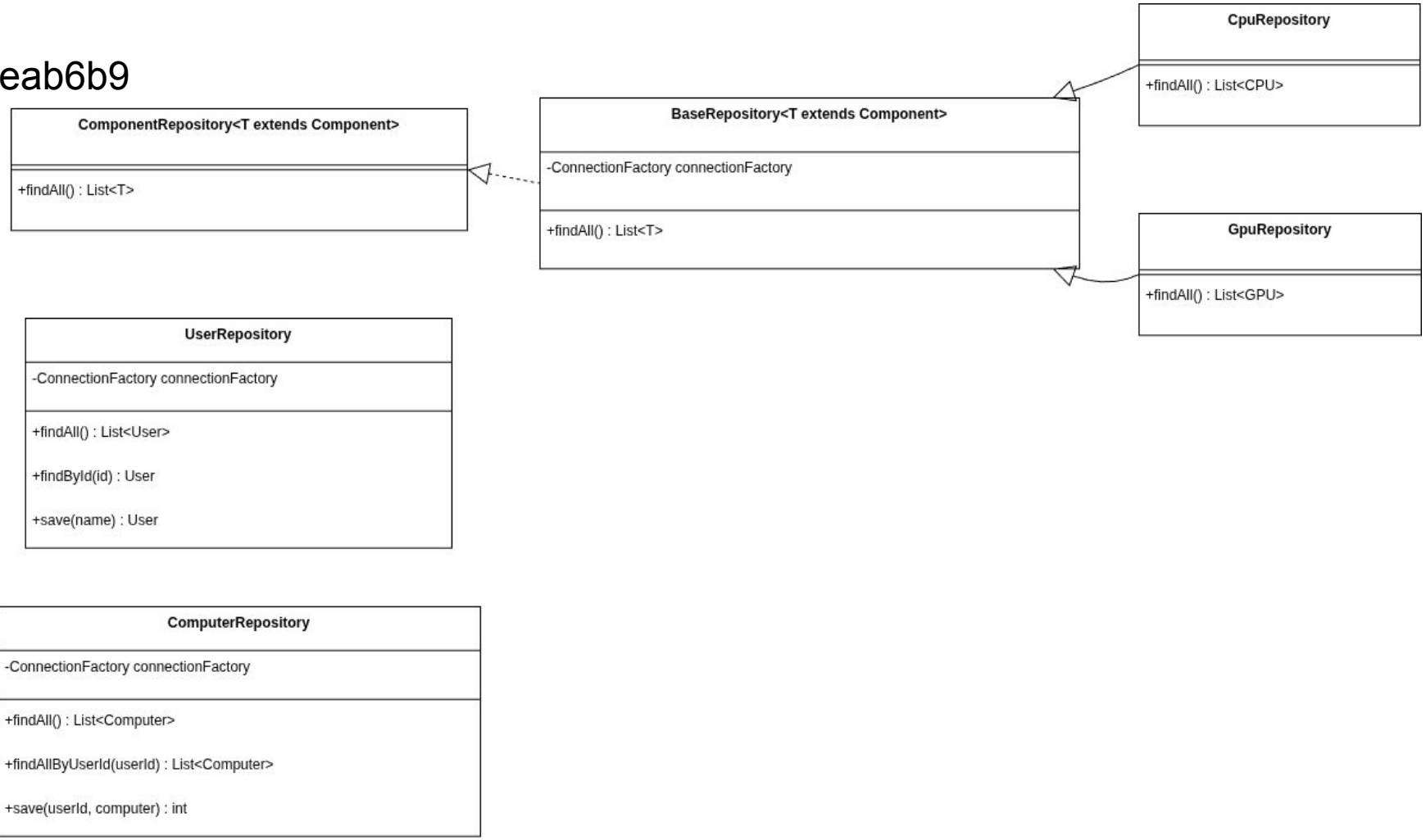


Nachher

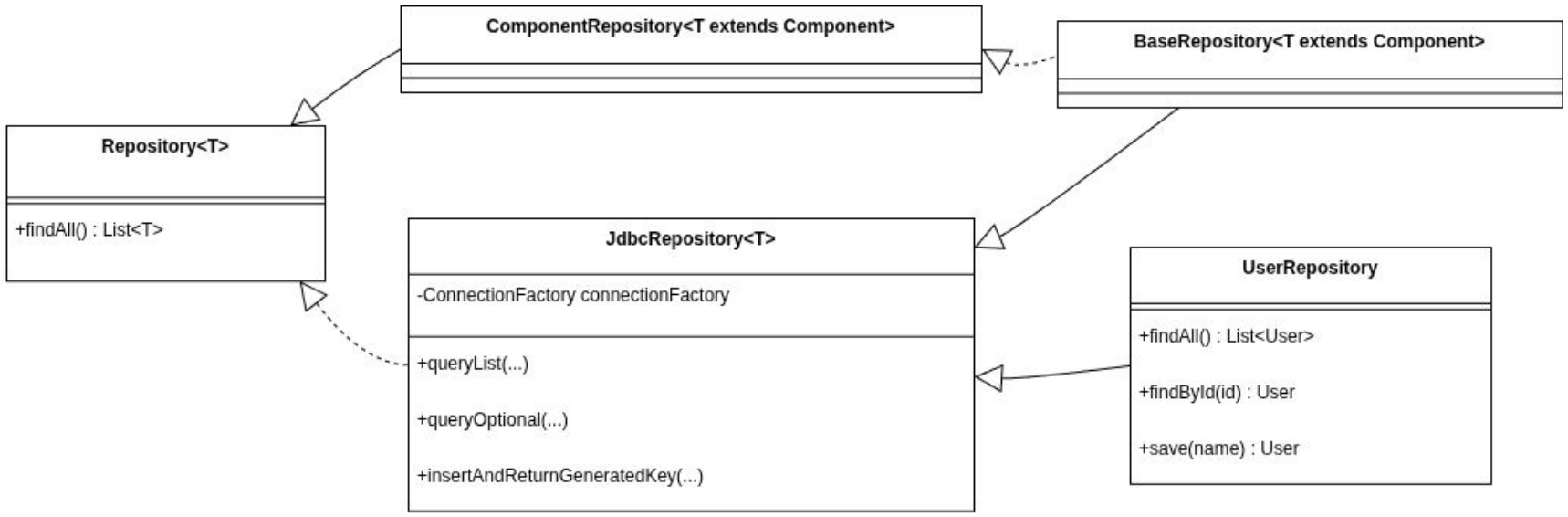


7.2 Refactoring - Duplicated Code (vorher)

Commit: beab6b9



7.2 Refactoring - Duplicated Code (nachher)

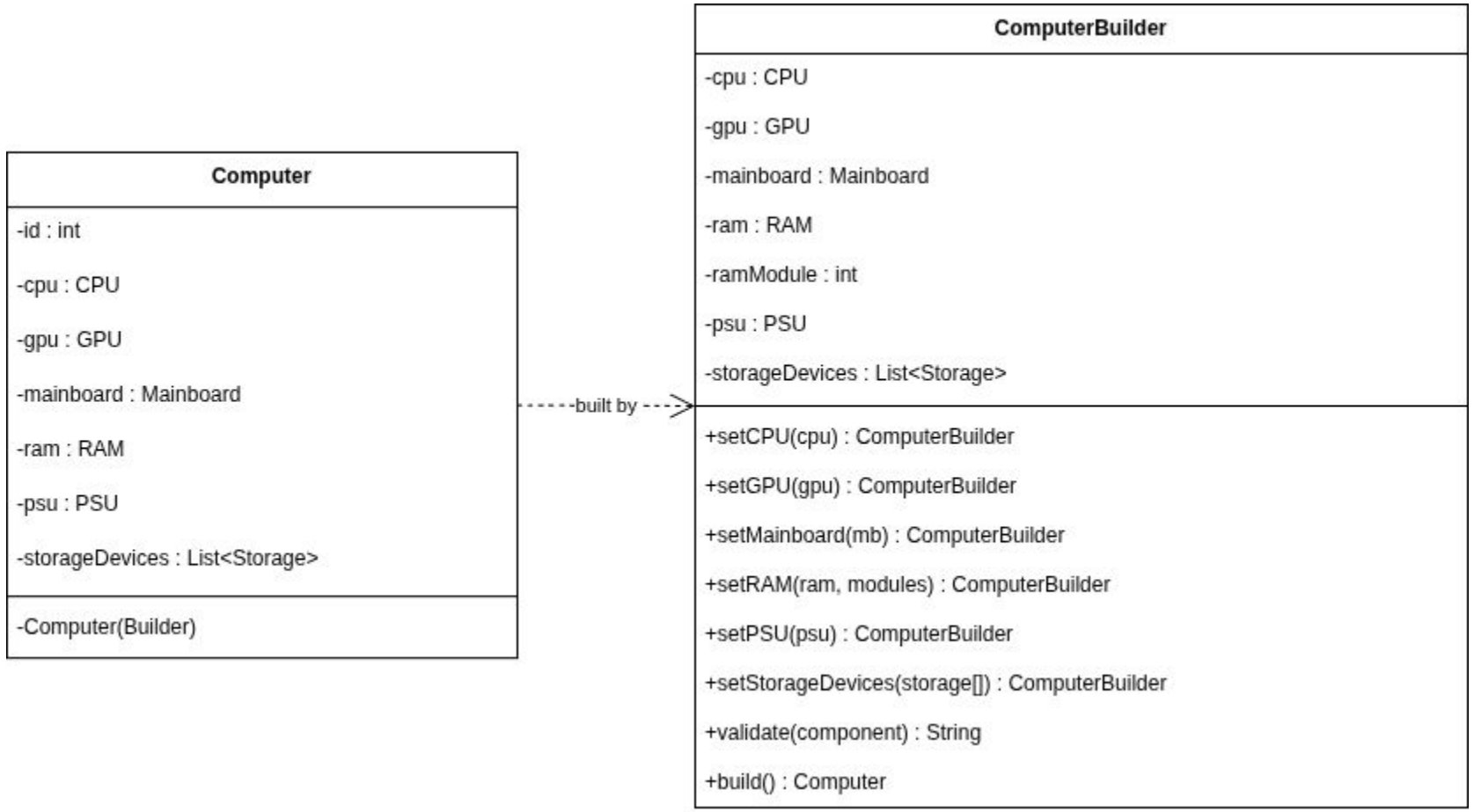


8. Entwurfsmuster - Builder

```java
public static class Builder {
    private CPU cpu;
    private GPU gpu;
    private Mainboard mainboard;
    private RAM ram;
    private int ramModule;
    private PSU psu;
    private List<Storage> storageDevices = new ArrayList<>();

    public Builder setCPU(CPU cpu) { this.cpu = cpu; return this; }
    public Builder setGPU(GPU gpu) { this.gpu = gpu; return this; }
    public Builder setRAM(RAM ram, int modules) { this.ram = ram; this.ramModule = modules; return this; }

    public String validate(Component component) { ... }

    public Computer build() { ... }
}
```

```java
// Verwendung im Ablauf
Computer newComputer = context.computerDraft.getBuilder().build();
int saveId = context.computerRepository.save(currentUser.getId(), newComputer);
```

8. Entwurfsmuster - Builder - UML



8. Entwurfsmuster - Composite

```java
public interface IMenuComponent {
    void execute();
    String getTitle();
    default boolean isVisible() { return true; }
}
```

```java
public class Menu implements IMenuComponent {
    private final List<IMenuComponent> children = new ArrayList<>();

    public void add(IMenuComponent component) {
        children.add(component);
    }

    @Override
    public void execute() {
        ...
        for (IMenuComponent child : children) {
            if (child.isVisible()) {
                System.out.println(child.getTitle());
            }
        }
        ...
    }
}
```

```java
public class MenuItem implements IMenuComponent {
    private final ICommand command;

    @Override
    public void execute() {
        command.execute();
    }
}
```

```java
// Aufbau in MenuFactory
Menu mainMenu = new Menu("PC Part Picker - Hauptmenü", context.inputReader);
mainMenu.add(new MenuItem("Computerverwaltung", new OpenMenuCommand(createComputerMenu())));
mainMenu.add(new MenuItem("Nutzerverwaltung", new OpenMenuCommand(createUserMenu())));
```

8. Entwurfsmuster - Composite - UML